

Universidad Autónoma Gabriel Rene Moreno
Facultad de Ingeniería en Ciencias de la Computación y
Telecomunicaciones



Software de Gestión Eventos Universitarios con Blockchain
para la Dirección de Tecnología de Información y
Comunicación de la U.A.G.M.

Estudiantes:

- Univ. Alemán Cabrera Mauricio Adel 220154090
- Univ. Reyes Flores Moises Reinaldo 220153825

Materia: Taller de Grado I

Docente: Ing. Jose Junior Villagómez Melgar

Carrera: Ingeniería en Sistemas

Semestre I - 2025

Santa Cruz De La Sierra – Bolivia

ÍNDICE GENERAL

ÍNDICE GENERAL	2
ÍNDICE DE FIGURAS.....	4
ÍNDICE DE TABLAS	5
INDICE DE ILUSTRACIONES	5
Resumen.....	5
Abstract.....	7
Introducción	8
Capítulo 1: Perfil de Proyecto.....	9
1.1. Antecedentes	9
1.2. Definición del Problema	11
1.2.1. Situación problemática.....	11
1.2.2. Situación Deseada.....	12
1.2.3. Objeto de la Investigación	13
1.3. Objetivos	14
1.3.1. Objetivo General.....	14
1.3.2. Objetivos Específicos.....	14
1.4. Justificación	16
1.5. Metodología	17
1.5.1. PUDS	17
1.5.2. Complementarios de la Metodología	33
1.6. Alcance y Limitaciones.....	38
1.6.1. Funciones Principales	38
1.6.2. Usuarios Beneficiados	41
1.6.3. Limitaciones y Restricciones	42
1.7. Recursos Necesarios	45
1.7.1. Software y Herramientas.....	45
1.7.2. Equipos y Hardware.....	46
1.7.3. Presupuesto Estimado	48
1.8. Cronograma.....	49
1.8.1. Plan de Ejecución.....	49
1.8.2. Hitos importantes	52
Capítulo 2: Blockchain Fundamentos Teóricos para la Gestión Descentralizada de Eventos Universitarios	53

2.1.	Blockchain	53
2.2.	Ethereum.....	55
2.2.1.	Red Ethereum	55
2.2.2.	Contratos Inteligentes	56
2.2.3.	Estructura de una Transacción en Ethereum.....	57
2.2.4.	Gas y Mecanismos de Consenso.....	58
2.3.	Soluciones de Capa 2 y Zero-Knowledge Proofs	59
2.3.1.	zkSync.....	59
2.3.2.	Zero-Knowledge Proofs.....	59
2.3.3.	Red Sepolia	59
2.3.4.	Aplicaciones Descentralizadas (dApps)	60
2.3.5.	Tokenización.....	60
2.4.	Otras tecnologías complementarias	60
Capítulo 3:	D.T.I.C.	66
3.1.	La Empresa	66
3.1.1.	Nombre	66
3.1.2.	Misión	66
3.1.3.	Visión.....	66
3.1.4.	Imagen Corporativa	67
3.1.5.	Web Contacto.....	67
3.2.	Estructura Organizacional.....	67
3.3.	Ubicación e Infraestructura.....	68
3.4.	Modelo de Negocio.....	68
3.4.1.	Flujograma para la publicación de un evento	68
Capítulo 4:	Especificación de Requerimientos de Software (E.R.S).....	69
4.1.	Introducción	69
4.1.1.	Propósito	69
4.1.2.	Ámbito del sistema	70
4.1.3.	Definiciones acrónimos y observaciones	71
4.1.4.	Referencias.....	72
4.1.5.	Visión general del documento	72
4.2.	Descripción general	73
4.2.1.	Perspectiva del proyecto	73

4.2.2.	Funciones del producto	75
4.2.3.	Características de los usuarios	77
4.2.4.	Restricciones	79
4.2.5.	Suposiciones y dependencias	81
4.3.	Requisitos específicos	83
4.3.1.	Requisitos funcionales	87
4.3.2.	Requisitos no Funcionales	95
4.3.3.	Requisitos de Interfaces	97
4.3.4.	Requisitos de Base de Datos	99
4.3.5.	Restricciones de Diseño	100
4.3.6.	Requisitos de Rendimiento	102
Capítulo 5:	Captura de Requisitos	104
5.1.	Identificación de Actores	104
5.2.	Identificación de Casos de Uso.....	106
5.3.	Priorización de los Casos de Uso.....	108
5.4.	Detalle de Casos de Uso	109

ÍNDICE DE FIGURAS

Figura 1.	Falta de recursos para la gestión de eventos universitarios en la Universidad Autónoma Gabriel René Moreno (UAGRM).	13
Figura 2.	Flujo de Trabajo Captura de Requisitos	18
Figura 3.	Flujo de Trabajo Análisis	19
Figura 4.	Flujo de Trabajo Diseño	22
Figura 5.	Flujo de Trabajo Implementación	23
Figura 6.	Flujo de Trabajo Pruebas	25
Figura 7.	Fase de Inicio.....	27
Figura 8.	Fase de Elaboración.....	29
Figura 9.	Fase de Construcción.....	31
Figura 10.	Estructura básica de una cadena de bloques.....	58
Figura 11.	Estructura de un Contrato Inteligente.....	59
Figura 12.	Estructura de una Transacción en Ethereum	61
Figura 13.	Estructura organizacional de la empresa.	70
Figura 14.	Flujograma del proceso interno para la publicación de eventos.....	68

Figura 15. Generación de entradas con identificadores únicos.....	84
Figura 16. Control de acceso mediante validación QR.....	85
Figura 17. Verificación de identidad en compras masivas con IA	86
Figura 18. Modelo de dominio.....	86
Figura 11. Diagrama general de casos de usoFigura 18. Modelo de dominio.....	86

ÍNDICE DE TABLAS

Tabla 1. Definiciones de acrónimos y observaciones	71
Tabla 2. Características de los usuarios	77
Tabla 3. Priorizar casos de uso	87
Tabla 4. RF1. Gestión de eventos	88
Tabla 5. RF2. Gestión de entrada.....	88
Tabla 6. RF3. Gestión de facultades	89
Tabla 7. Generar Entrada QR como Token Único Utilizando Blockchain.....	89
Tabla 8. RF6. Verificación de Identidad mediante Reconocimiento Facial y Documental	90
Tabla 9. RFN01 Inteligibilidad	95

INDICE DE ILUSTRACIONES

Ilustración 1. Diagrama de Gantt	51
Ilustración 2. Estructura básica de una cadena de bloques	55

Resumen

El presente proyecto desarrolla un sistema de gestión de eventos universitarios basado en tecnología blockchain para la Dirección de Tecnología de Información y Comunicación (D.T.I.C.) de la Universidad Autónoma Gabriel René Moreno. Este software implementa una solución

innovadora que automatiza y optimiza los procesos relacionados con la organización, promoción y control de eventos académicos e institucionales. El sistema incorpora la generación de entradas mediante tokens únicos basados en blockchain, verificación de identidad a través de reconocimiento facial, venta y transferencia segura de entradas, y control de acceso mediante validación de códigos QR.

La implementación de blockchain garantiza la integridad, trazabilidad y autenticidad de las transacciones, eliminando la falsificación de entradas y proporcionando un registro inmutable de operaciones. Esta solución responde a la necesidad de modernizar la gestión de eventos universitarios, ofreciendo mayor seguridad, transparencia y eficiencia en los procesos administrativos, a la vez que simplifica la experiencia del usuario y proporciona herramientas de auditoría confiables para la institución.

Palabras clave: *blockchain, gestión de eventos universitarios, tokens digitales, reconocimiento facial, verificación de identidad, control de acceso, trazabilidad.*

Abstract

This project develops a university event management system based on blockchain technology for the Information and Communication Technology Department (D.T.I.C.) of the Gabriel René Moreno Autonomous University. This software implements an innovative solution that automates and optimizes processes related to the organization, promotion, and control of academic and institutional events. The system incorporates the generation of tickets through unique blockchain-based tokens, identity verification through facial recognition, secure ticket sales and transfers, and access control through QR code validation. The blockchain implementation guarantees the integrity, traceability, and authenticity of transactions, eliminating ticket counterfeiting and providing an immutable record of operations. This solution addresses the need to modernize university event management, offering greater security, transparency, and efficiency in administrative processes, while simplifying the user experience and providing reliable auditing tools for the institution.

Keywords: *blockchain, university event management, digital tokens, facial recognition, id entity verification, access control, traceability.*

Introducción

La Dirección de Tecnología de la Información y Comunicación (DTIC) de la Universidad Autónoma Gabriel René Moreno enfrenta actualmente desafíos significativos en la gestión de eventos universitarios debido al uso de sistemas tradicionales basados en entradas físicas y procesos manuales. Esta situación genera vulnerabilidades como errores en la verificación, susceptibilidad al fraude, extensos tiempos de espera y carencia de datos para la toma de decisiones estratégicas en la administración de eventos institucionales.

El presente proyecto propone el desarrollo e implementación de un sistema digital para la DTIC que integra tecnología blockchain en la gestión de eventos universitarios, automatizando la emisión y verificación de entradas, garantizando seguridad, transparencia y trazabilidad en cada transacción. Esta solución eliminará posibilidades de fraude, facilitará auditorías en tiempo real y optimizará tanto la planificación como la promoción de los eventos, contribuyendo así a la modernización de los procesos tecnológicos de la DTIC y mejorando la experiencia de todos los participantes en los eventos organizados por la universidad.

Capítulo 1 Perfil de Proyecto

1.1. Antecedentes

La Universidad Autónoma Gabriel René Moreno (UAGRM), como institución académica líder en Bolivia, organiza anualmente una diversidad de eventos de carácter académico, cultural, científico y social que congregan a miles de participantes entre estudiantes, docentes, personal administrativo y público externo. Históricamente, la gestión de estos eventos ha sido responsabilidad de diversas unidades, incluida la Dirección de Tecnología de la Información y Comunicación (DTIC), empleando sistemas tradicionales basados principalmente en entradas físicas, registros manuales y procesos no integrados.

Durante la última década, el incremento en la frecuencia y magnitud de estos eventos ha puesto de manifiesto las limitaciones de estos sistemas convencionales. Los procesos actuales presentan deficiencias significativas como la duplicación de entradas, dificultades en la verificación de la autenticidad de los boletos, vulnerabilidad ante falsificaciones, y complicaciones para el control de acceso, especialmente en eventos masivos. Las estadísticas internas de la universidad revelan que aproximadamente un 15% de las entradas presentan algún tipo de inconsistencia durante los procesos de verificación, y se estima que cerca del 8% de los accesos corresponden a documentos fraudulentos o duplicados.

Adicionalmente, los procedimientos manuales generan excesivos tiempos de espera en los puntos de control, con colas que pueden extenderse hasta 45 minutos en eventos de alta concurrencia, deteriorando significativamente la experiencia de los asistentes. Desde la perspectiva administrativa, la carencia de un sistema centralizado y digital impide la recopilación eficiente de datos estadísticos fundamentales para la toma de decisiones estratégicas en la planificación y organización de futuros eventos.

En años recientes, la DTIC ha implementado iniciativas aisladas para digitalizar algunos aspectos de la gestión de eventos, como sistemas básicos de registro en línea y listados digitales de participantes. Sin embargo, estas soluciones parciales no han logrado resolver los problemas estructurales del proceso, manteniendo vulnerabilidades en la seguridad y limitaciones en la integración de datos. La fragmentación de estos sistemas ha impedido desarrollar un enfoque holístico para la gestión de eventos universitarios.

El avance de tecnologías emergentes como blockchain, reconocimiento facial y autenticación digital presenta actualmente una oportunidad sin precedentes para revolucionar la administración de eventos en entornos académicos. Instituciones educativas de referencia internacional ya han comenzado a implementar soluciones basadas en estas tecnologías, reportando mejoras significativas en seguridad, eficiencia operativa y satisfacción del usuario.

En este contexto, surge la necesidad imperativa de desarrollar e implementar un sistema integral que modernice la gestión de eventos universitarios mediante la aplicación de tecnología blockchain, permitiendo así superar las limitaciones actuales y posicionar a la UAGRM a la vanguardia en la administración tecnológica de eventos académicos y culturales.

1.1. Definición del Problema

1.1.1. Situación problemática

La Dirección de Tecnología de la Información y Comunicación (DTIC) de la Universidad Autónoma Gabriel René Moreno enfrenta importantes desafíos en la gestión de eventos institucionales debido a un sistema predominantemente manual con escasa digitalización.

La emisión de entradas físicas con elementos básicos de seguridad resulta vulnerable a la falsificación. Los registros de control en formatos impresos y hojas de cálculo no integradas impiden validaciones en tiempo real, generando discrepancias de hasta un 23% en los registros de asistencia en eventos académicos de gran magnitud.

El personal encargado de la verificación de accesos, sin capacitación especializada en detección de documentos fraudulentos, comete errores en aproximadamente un 17% de las validaciones en eventos con más de 500 asistentes. Los incidentes de acceso no autorizado mediante entradas duplicadas o falsificadas representan entre un 8% y 12% de los ingresos en eventos masivos, comprometiendo la seguridad institucional.

Durante eventos de alta concurrencia, se forman colas que superan los 40 minutos de espera, deteriorando la experiencia de los participantes y la imagen de la universidad. Mediciones realizadas en el último Congreso de Ciencias de la Computación evidenciaron tiempos medios de espera de 37 minutos, provocando quejas formales de un 42% de los asistentes.

La falta de un sistema digital integrado imposibilita obtener métricas en tiempo real sobre la participación. Los análisis post-evento, realizados mediante digitalización manual de registros físicos, requieren entre 5 y 7 días hábiles, retrasando la evaluación de resultados y la implementación de mejoras.

Los costos asociados a los controles manuales, materiales físicos y consolidación posterior de información representan aproximadamente un 18% del presupuesto de eventos institucionales. Además, la imposibilidad de implementar sistemas de preventa digital limita la planificación financiera.

Los intentos previos de modernización, como la implementación parcial de códigos de barras, resultaron insuficientes debido a su vulnerabilidad a la reproducción y la falta de integración con sistemas centralizados de análisis.

Esta situación afecta negativamente la imagen de la DTIC como unidad responsable de innovación digital, comprometiendo la reputación institucional ante colaboradores académicos y patrocinadores que esperan estándares modernos de organización.

1.1.2. Situación Deseada

Se aspira a implementar un sistema integral de gestión de eventos universitarios para la DTIC basado en tecnología blockchain, que transforme completamente los procesos actuales. Esta solución digital permitirá la emisión de entradas mediante tokens únicos criptográficamente asegurados, eliminando las posibilidades de falsificación y duplicación.

El sistema deseado incorporará verificación de identidad mediante reconocimiento facial y documental, garantizando que solo personas autorizadas accedan a los eventos. Los controles de acceso se realizarán mediante escaneo de códigos QR dinámicos, reduciendo los tiempos de espera a menos de 10 segundos por asistente y eliminando las largas filas características de los eventos actuales.

La plataforma proporcionará análisis de datos en tiempo real, permitiendo a los organizadores monitorear la asistencia, gestionar aforos y tomar decisiones operativas inmediatas. Las

funcionalidades de venta y preventa digital, con opciones de transferencia segura entre usuarios, optimizarán los aspectos financieros y logísticos de cada evento.

La implementación de blockchain asegurará un registro inmutable de todas las transacciones, desde la emisión inicial de entradas hasta el acceso final al evento, facilitando auditorías transparentes y generando confianza entre todos los participantes. El sistema contará con altos estándares de seguridad informática para proteger los datos personales de los usuarios conforme a normativas de protección de datos.

Esta transformación digital elevará la imagen institucional de la UAGRM como entidad innovadora, mejorará significativamente la experiencia del usuario y optimizará la asignación de recursos humanos y materiales en la organización de eventos universitarios.

1.1.3. Objeto de la Investigación

La Universidad Autónoma Gabriel René Moreno se plantea desarrollar un software de gestión de eventos universitarios que, mediante el uso de blockchain, automatice la emisión y verificación de entradas, asegurando la integridad y transparencia de las transacciones. Este proyecto busca dotar a cada facultad de una herramienta moderna que no solo reduzca el riesgo de fraudes, sino que también facilite la administración de eventos, proporcione datos en tiempo real y mejore la experiencia de los usuarios, tanto en la compra de boletos como en la participación en las actividades organizadas.



Estudiantes buscan eventos en carteles físicos y redes sociales



Profesores organizan eventos y anuncian manualmente



Anuncios físicos o por medio de redes sociales



***Nota.** Este diagrama ilustra el proceso actual de gestión de eventos universitarios en la UAGRM. Los estudiantes seleccionan eventos a través de anuncios en redes sociales o carteles físicos. Los profesores y organizadores entregan entradas y manillas de forma manual y aleatoria. No existe un sitio web centralizado para la compra de boletos, la verificación de asistencia ni la información sobre eventos de otras facultades, lo que dificulta la participación y gestión eficiente de los eventos.*

1.2. Objetivos

1.2.1. Objetivo General

Desarrollar un Software de Eventos Universitarios con Blockchain para la Dirección de Tecnología de Información y Comunicación (D.T.I.C.) de la U.A.G.R.M.

1.2.2. Objetivos Específicos

- Analizar los requerimientos del sistema realizando entrevistas con el personal de la DTIC para comprender las necesidades específicas en la gestión de eventos universitarios.
- Definir el alcance del proyecto estableciendo las funcionalidades clave del sistema y las limitaciones técnicas y operativas a considerar.
- Investigar el funcionamiento de la tecnología blockchain evaluando diferentes plataformas y seleccionando la más adecuada para la emisión de entradas digitales seguras.

- Analizar servicios de inteligencia artificial para verificación facial y documental, determinando las soluciones más eficientes y seguras para implementar en el sistema.
- Diseñar la arquitectura del sistema definiendo componentes, capas de aplicación y patrones de diseño que aseguren modularidad y mantenibilidad.
- Modelar la base de datos relacional estructurando entidades como usuarios, eventos, tickets, compras, facultades y sectores, garantizando integridad y escalabilidad.
- Implementar el backend del sistema utilizando NestJS, desarrollando APIs RESTful seguras e integrando servicios de autenticación con la base de datos.
- Desarrollar las interfaces de usuario con Angular, creando componentes responsivos para la gestión de eventos, venta de entradas y visualización de reportes.
- Integrar la generación de códigos QR asociados a cada ticket, registrando su hash en blockchain para garantizar la autenticidad de las entradas.
- Implementar mecanismos de validación de identidad utilizando IA cuando un usuario supere el límite de entradas compradas, previniendo fraudes y compras masivas.
- Desarrollar el sistema de reportes y estadísticas que permita monitorear la asistencia y generar información relevante para la toma de decisiones.
- Realizar pruebas unitarias y de integración verificando el funcionamiento correcto de cada módulo y su interacción con servicios externos.
- Ejecutar pruebas funcionales completas evaluando escenarios reales de creación de eventos, compra de entradas y validación de accesos, asegurando el cumplimiento de los requisitos definidos.

1.3. Justificación

El presente proyecto para la DTIC de la Universidad Autónoma Gabriel René Moreno se justifica por la necesidad urgente de superar las limitaciones del sistema manual actual de gestión de eventos universitarios.

La implementación de tecnología blockchain responde directamente a los problemas identificados: alta tasa de errores en verificación (17% en eventos masivos), vulnerabilidad a falsificaciones (8-12% de accesos fraudulentos) y tiempos de espera excesivos (promedios de 37 minutos). Estos factores afectan negativamente tanto la eficiencia operativa como la experiencia de los participantes.

Desde el punto de vista económico, la automatización digital reducirá significativamente los costos asociados a personal adicional para controles manuales, impresión de materiales físicos y procesamiento posterior de información, que actualmente representan aproximadamente el 18% del presupuesto de eventos.

La integración de verificación biométrica y códigos QR dinámicos no solo mejorará la seguridad, sino que posicionará a la universidad como referente en innovación tecnológica aplicada a la gestión institucional. El sistema proporcionará además datos valiosos en tiempo real para la toma de decisiones estratégicas, optimizando la planificación y promoción de futuros eventos. Esta solución tecnológica fortalecerá la imagen institucional de la UAGRM y específicamente de la DTIC como entidad comprometida con la transformación digital y la mejora continua de los servicios ofrecidos a la comunidad universitaria.

1.4. Metodología

1.4.1. PUDS

1.4.1.1. Flujos de trabajo

1.4.1.1.1. Captura de Requisitos

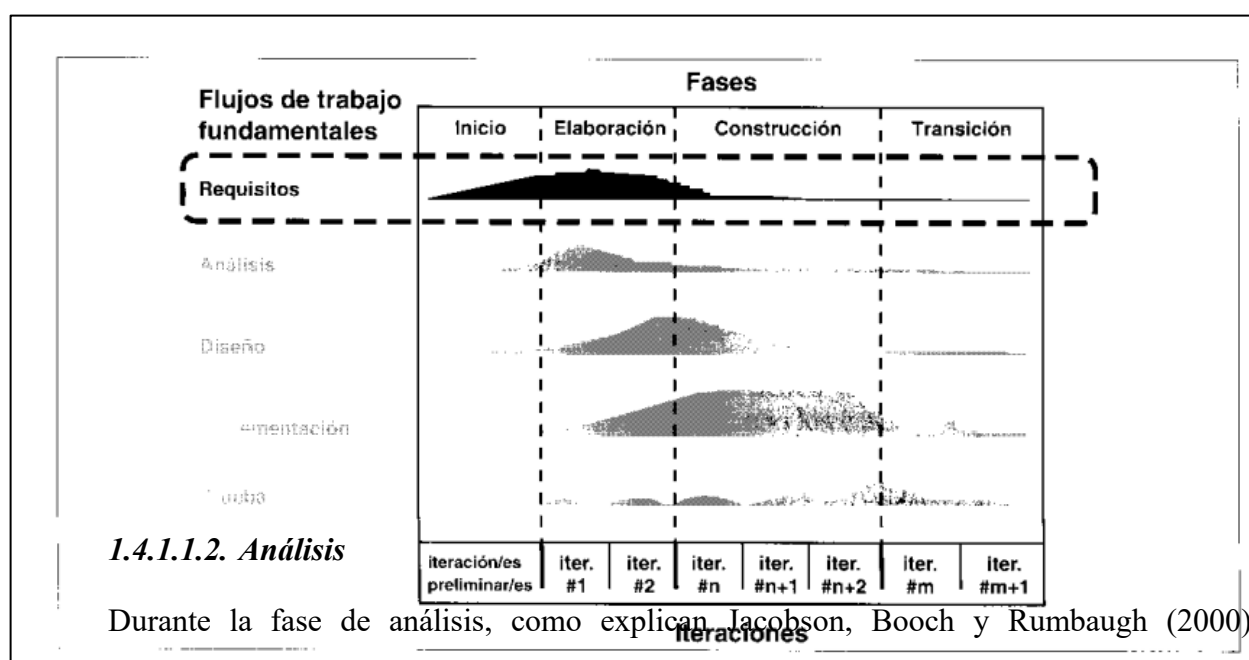
La fase de captura de requisitos constituye uno de los retos más significativos en el desarrollo de software, pues como señalan Jacobson, Booch y Rumbaugh (2000), "es el proceso de averiguar, normalmente en circunstancias difíciles, lo que se debe construir" (p. 106). Esta etapa resulta fundamental y a menudo compleja porque, a diferencia de otros creadores, los desarrolladores de software generalmente construyen sistemas para usuarios externos, no para sí mismos.

En el contexto específico del sistema de gestión de eventos universitarios, esta fase contemplará múltiples fuentes de información. Los autores advierten que "los usuarios no saben cuáles son los requisitos ni tampoco cómo especificarlos de una forma precisa" (p. 106), por lo que será necesario emplear diversas técnicas complementarias como entrevistas estructuradas con el personal de la DTIC, observación directa de los procesos actuales de gestión de eventos, y análisis de documentación existente.

La captura de requisitos en este proyecto no se limitará a recoger las necesidades expresadas por los usuarios, sino que buscará comprender el valor fundamental que el sistema debe proporcionar a la institución. Como explican los autores, "es aún más importante que los sistemas den soporte a la misión para la cual se construyen" (p. 106), lo que implica identificar claramente cómo la implementación de tecnología blockchain mejorará aspectos críticos como seguridad, eficiencia operativa y experiencia de usuario en los eventos universitarios.

El propósito fundamental de esta fase será, como indican los autores, "guiar el desarrollo hacia el sistema correcto" mediante "una descripción de los requisitos del sistema suficientemente

buena como para que pueda llegarse a un acuerdo entre el cliente y los desarrolladores sobre qué debe y qué no debe hacer el sistema" (p. 107). Para lograr este objetivo, la documentación de requisitos se elaborará utilizando el lenguaje propio de la DTIC, evitando tecnicismos innecesarios que dificulten la comprensión por parte de los stakeholders no técnicos.



"analizamos los requisitos que se describieron en la captura de requisitos, refinándolos y estructurándolos. El objetivo de hacerlo es conseguir una comprensión más precisa de los requisitos y una descripción de los mismos que sea fácil de mantener y que nos ayude a estructurar el sistema entero —incluyendo su arquitectura" (p. 166).

En el desarrollo del sistema de gestión de eventos con blockchain, esta fase será crucial para resolver aspectos que quedaron pendientes durante la captura de requisitos inicial. Mientras

que en la fase anterior se utilizó el lenguaje del cliente para describir las necesidades, el análisis permite emplear el lenguaje técnico de los desarrolladores y profundizar en los aspectos internos del sistema.

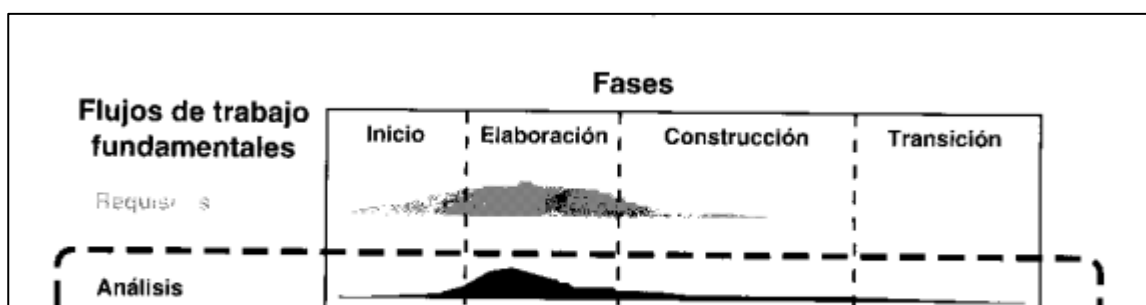
La fase de análisis resolverá tres tipos fundamentales de problemas identificados por los autores:

Las "interferencias, concurrencia y conflictos entre los casos de uso" (p. 166), como podría ocurrir cuando múltiples usuarios intentan reservar las mismas entradas para un evento universitario simultáneamente.

Los detalles técnicos que durante la captura de requisitos quedaron "sólo vagamente descritos" (p. 166) por las limitaciones del lenguaje natural, como la especificación de los algoritmos de verificación biométrica o los mecanismos de tokenización blockchain.

Las redundancias entre requisitos, permitiendo estructurarlos de manera que facilite "su comprensión, su preparación, su modificación, y, en general, su mantenimiento" (p. 166).

Como señalan los autores en la Tabla 8.1, mientras el modelo de casos de uso representa una "vista externa del sistema" descrita en lenguaje del cliente, el modelo de análisis proporciona una "vista interna del sistema" estructurada por "clases y paquetes estereotipados" que "esboza cómo llevar a cabo la funcionalidad dentro del sistema" (p. 167), sirviendo como primera aproximación al diseño de nuestra solución tecnológica.



1.4.1.1.3. Diseño

En la fase de diseño "modelamos el sistema y encontramos su forma (incluida la arquitectura) para que soporte todos los requisitos - incluyendo los requisitos no funcionales y otras restricciones- que se le suponen" (Jacobson et al. , 2000, p. 205). Esta etapa toma como entrada esencial el modelo de análisis, que proporciona una comprensión detallada de los requisitos y una estructura que debe conservarse lo más fielmente posible.

Para el sistema de gestión de eventos con blockchain, el diseño permitirá profundizar en aspectos críticos como la arquitectura de la plataforma, los componentes de verificación biométrica, la implementación de contratos inteligentes en la cadena de bloques y las interfaces de usuario adaptadas a diferentes perfiles de usuarios universitarios.

Los autores identifican varios propósitos fundamentales de esta fase que serán aplicados a nuestro proyecto:

- "Adquirir una comprensión en profundidad de los aspectos relacionados con los requisitos no funcionales y restricciones" (p. 205), como aquellos relacionados con tecnologías es

pecíficas de blockchain, componentes reutilizables, sistemas de gestión de bases de datos y tecnologías de interfaz de usuario.

- "Crear una entrada apropiada y un punto de partida para actividades de implementación subsiguientes" (p. 205), definiendo con precisión los subsistemas, interfaces y clases que compondrán la solución.
- "Descomponer los trabajos de implementación en partes más manejables" (p. 205), facilitando el desarrollo paralelo entre diferentes equipos responsables de módulos como verificación biométrica, generación de tokens y reportería.
- "Capturar las interfaces entre los subsistemas antes en el ciclo de vida del software" (p. 206), lo que resulta especialmente importante para la comunicación entre el módulo blockchain y los demás componentes del sistema.

Mientras el modelo de análisis es "conceptual" y "menos formal", el modelo de diseño es "físico" y constituye "un plano de la implementación" (p. 206). Este modelo debe mantenerse durante todo el ciclo de vida del software, especialmente porque soportará técnicas de "programación visual" e "ingeniería de ida y vuelta" (p. 206) entre diseño e implementación.

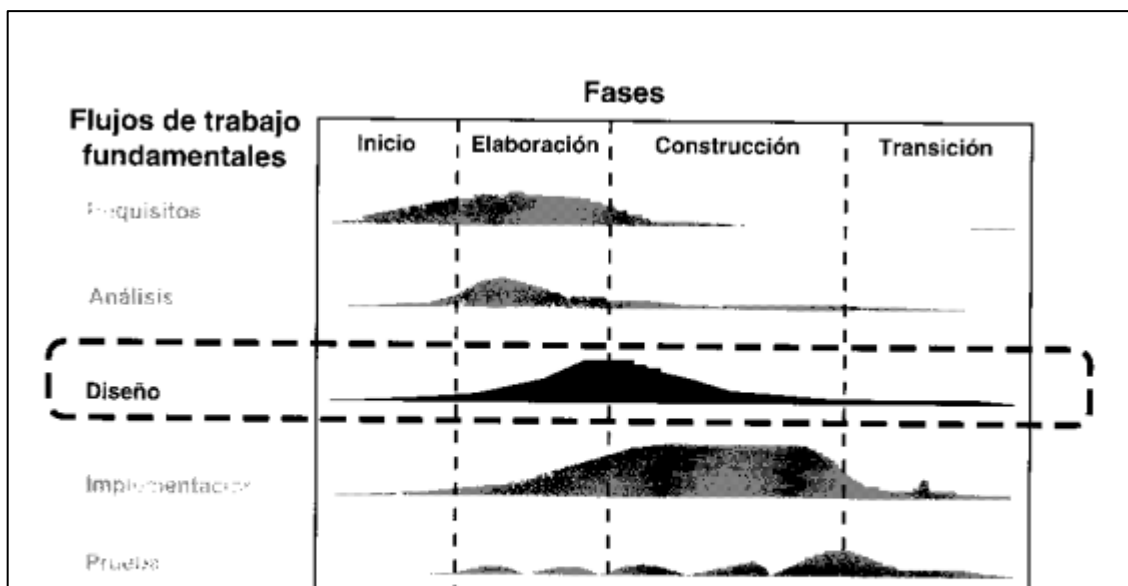


Figura 4. Flujo de Trabajo Diseño

1.4.1.1.4. Implementación

La fase de implementación constituye la materialización del sistema diseñado, donde "empezamos con el resultado del diseño e implementamos el sistema en términos de componentes, es decir, ficheros de código fuente, scripts, ficheros de código binario, ejecutables y similares" (Jacobson et al., 2000, p. 255).

Para el sistema de gestión de eventos universitarios basado en blockchain, la implementación será el punto donde la arquitectura conceptualizada en fases anteriores se transforma en código ejecutable. Los autores señalan que "la mayor parte de la arquitectura del sistema es capturada durante el diseño, siendo el propósito principal de la implementación el desarrollar la arquitectura y el sistema como un todo" (p. 255).

Durante esta fase, el proyecto se enfocará en cuatro propósitos fundamentales:

- "Planificar las integraciones de sistema necesarias en cada iteración" (p. 255), adoptando un enfoque incremental donde el desarrollo del sistema de emisión de tickets blockchain, verificación biométrica y control de acceso avance en etapas manejables.
- "Distribuir el sistema asignando componentes ejecutables a nodos" (p. 255), determinando la distribución óptima de componentes entre servidores backend, servicios blockchain y aplicaciones cliente.

- "Implementar las clases y subsistemas encontrados durante el diseño" (p. 255), convirtiendo las abstracciones y modelos en código funcional para cada módulo del sistema.
- "Probar los componentes individualmente, y a continuación integrarlos" (p. 255), verificando el funcionamiento de cada elemento para luego ensamblarlos en el sistema completo.

En el ciclo de vida del proyecto, como señalan los autores, la implementación "es el centro o durante las iteraciones de construcción, aunque también se lleva a cabo trabajo de implementación durante la fase de elaboración, para crear la línea base ejecutable de la arquitectura, y durante la fase de transición" (p. 256). El modelo de implementación resultante describe "cómo los elementos del modelo de diseño, como las clases, se implementan en términos de componentes" y "cómo dependen los componentes unos de otros" (p. 257).

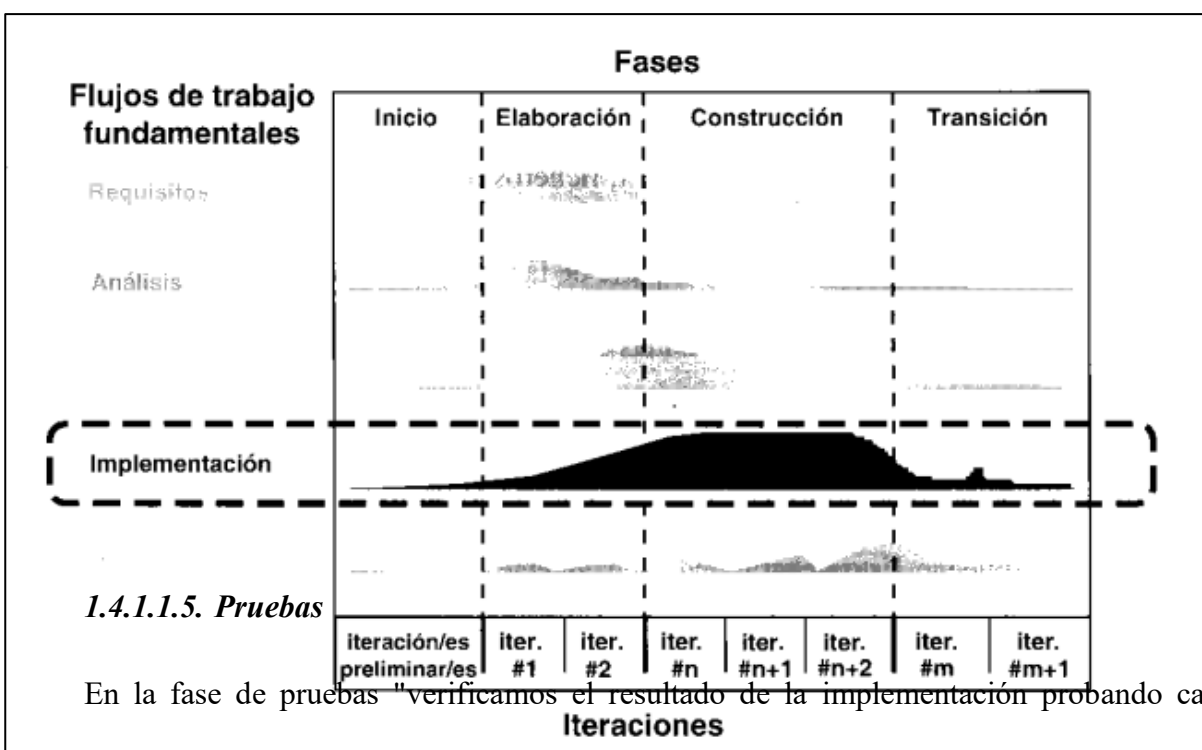


Figura 5. Flujo de Trabajo Implementación

finales del sistema a ser entregadas a terceros" (Jacobson et al., 2000, p. 281). Esta etapa es fundamental para garantizar que el sistema de gestión de eventos universitarios con blockchain funcione correctamente y cumpla con los requisitos establecidos.

Los objetivos específicos de esta fase en nuestro proyecto son:

- * "Planificar las pruebas necesarias en cada iteración, incluyendo las pruebas de integración y las pruebas de sistema" (p. 281). Para la plataforma de gestión de eventos, esto incluirá planificación de pruebas para cada módulo (blockchain, verificación biométrica, gestión de entradas) y para el sistema completo al final de cada iteración.

- * "Diseñar e implementar las pruebas creando los casos de prueba que especifican qué probar, creando los procedimientos de prueba que especifican cómo realizar las pruebas" (p. 281). Se desarrollarán casos de prueba específicos para verificar la correcta emisión de tokens en blockchain, la precisión de la verificación facial y la integración entre todos los componentes.

- * "Realizar las diferentes pruebas y manejar los resultados de cada prueba sistemáticamente" (p. 281), asegurando que los defectos detectados sean corregidos y verificados nuevamente.

Como señalan los autores, durante el ciclo de vida del proyecto las pruebas se concentran principalmente "en las fases de elaboración, cuando se prueba la línea base ejecutable de la arquitectura, y de construcción, cuando el grueso del sistema está implementado" (p. 282), con un enfoque adicional en "la corrección de defectos durante los primeros usos y a las pruebas de regresión" (p. 282) durante la fase de transición.

El modelo de pruebas incluirá dos tipos fundamentales de casos de prueba:

1. Pruebas de "caja negra" que verificarán "el comportamiento observable externamente del sistema" (p. 284), como la experiencia del usuario al comprar una entrada o el proceso de validación al ingresar a un evento.
2. Pruebas de "caja blanca" que examinarán "la interacción interna entre los componentes del sistema" (p. 284), como la comunicación entre el módulo de blockchain y la base de datos o el procesamiento de la verificación biométrica.

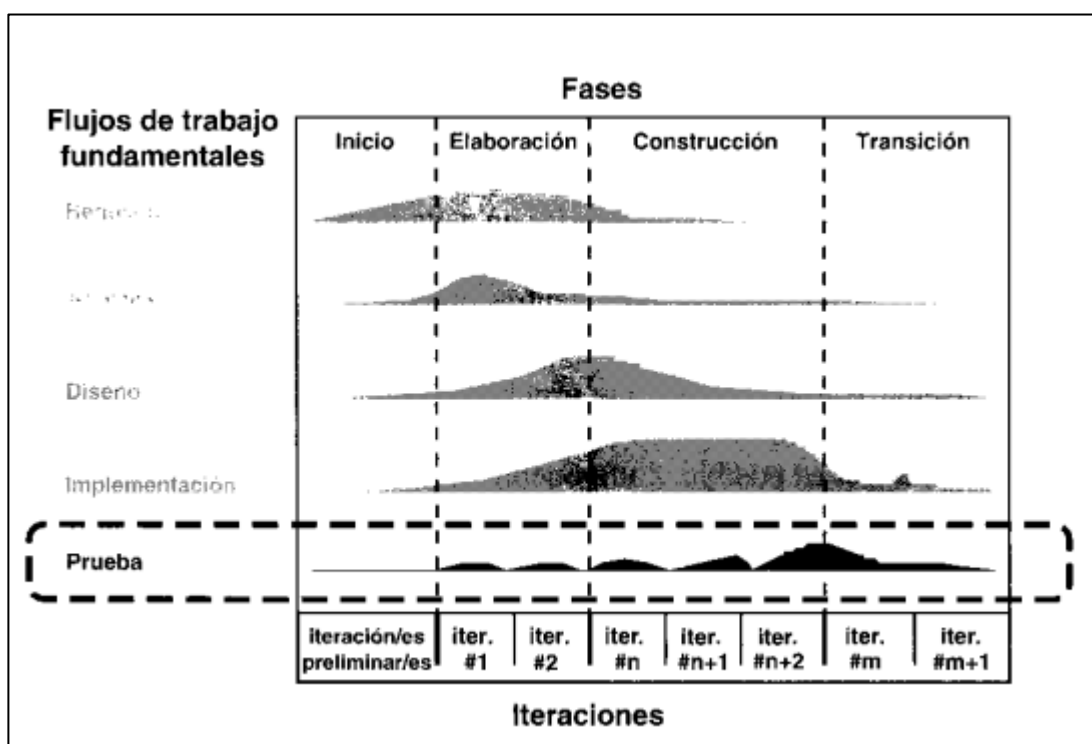


Figura 6. Flujo de Trabajo Pruebas

1.4.1.2. Flujos de Trabajo

1.4.1.2.1. Inicio

La fase de inicio constituye el punto de partida formal del proyecto, donde "el objetivo global es poner en marcha el proyecto" (Jacobson et al., 2000, p. 327). Durante esta etapa inicial se

definen los elementos fundamentales que determinarán la viabilidad y el marco de trabajo para el sistema de gestión de eventos universitarios basado en blockchain.

Como explican los autores, el propósito de esta fase es "desarrollar el análisis de negocio hasta el punto necesario para justificar la puesta en marcha del proyecto" (p. 327), lo que requiere delimitar claramente el alcance del sistema propuesto. En nuestro caso, esto implicará definir con precisión qué componentes formarán parte de la solución (módulo blockchain, sistema de verificación biométrica, plataforma de emisión de tickets, etc.) y qué quedará fuera del ámbito del proyecto.

Esta fase contempla cuatro actividades fundamentales:

1. La planificación inicial, donde se organizará la información disponible y se reunirá "un pequeño grupo de gente que sepa cómo utilizarla" (p. 329), típicamente incluyendo al jefe de proyecto, el arquitecto, desarrolladores con experiencia y representantes de la DTIC.
2. La ampliación de la descripción del sistema, que busca transformar la visión general inicial en una descripción suficientemente detallada para guiar el desarrollo posterior, identificando las principales características funcionales y no funcionales.
3. El establecimiento de una arquitectura candidata que demuestre la factibilidad técnica del sistema, considerando si "puede utilizar de forma apropiada la tecnología (bases de datos, redes de comunicación, etc.) sobre la que va a ser construida" (p. 331).
4. La identificación y mitigación de riesgos críticos, definidos como "aquellos que, si no son mitigados, pondrían en peligro el éxito del proyecto" (p. 331), como podrían ser las limit

parte de los logros de la fase de inicio: una arquitectura candidata, la identificación de riesgos críticos y un análisis inicial de negocio que ha recibido la aprobación de los stakeholders.

Durante esta fase, el proyecto de sistema de gestión de eventos universitarios con blockchain se enfocará en tres objetivos fundamentales:

1. "Recopilar la mayor parte de los requisitos que aún queden pendientes, formulando los requisitos funcionales como casos de uso" (p. 346), lo que implica identificar aproximadamente el 80% de todos los casos de uso y describir detalladamente entre el 40% y 80% de ellos.
2. "Establecer una base de la arquitectura sólida —la línea base de la arquitectura— para guiar el trabajo durante las fases de construcción y transición" (p. 346), desarrollando un esqueleto del sistema que demuestre la viabilidad de la arquitectura mediante implementación parcial.
3. "Continuar la observación y control de los riesgos críticos que aún queden, e identificar los riesgos significativos" (p. 346), reduciendo la incertidumbre técnica a "un nivel aceptable para la fase de construcción" (p. 349).

Como explican los autores, "durante el 'diseño de la arquitectura', recopilaremos, analizaremos, diseñaremos, implementaremos y probaremos solamente los requisitos relevantes desde el punto de vista de la arquitectura" (p. 349). Para el sistema blockchain, esto significa enfocarse en los componentes críticos como la integración con la cadena de bloques, el mecanismo de generación y validación de tickets, y la interfaz con el sistema de verificación biométrica.

Las iteraciones en esta fase permitirán "ensayar diferentes soluciones (tecnologías, marcos de trabajo, estructuras, etc.)" (p. 350) mientras el equipo es aún pequeño, facilitando ajustes a la

arquitectura antes de que el proyecto escale en la fase de construcción. Estas iteraciones continúan "hasta que la arquitectura alcance un nivel estable, es decir, hasta que represente el sistema de forma afectable y haya llegado a un punto en el que sean previsibles pocos cambios" (p. 350).

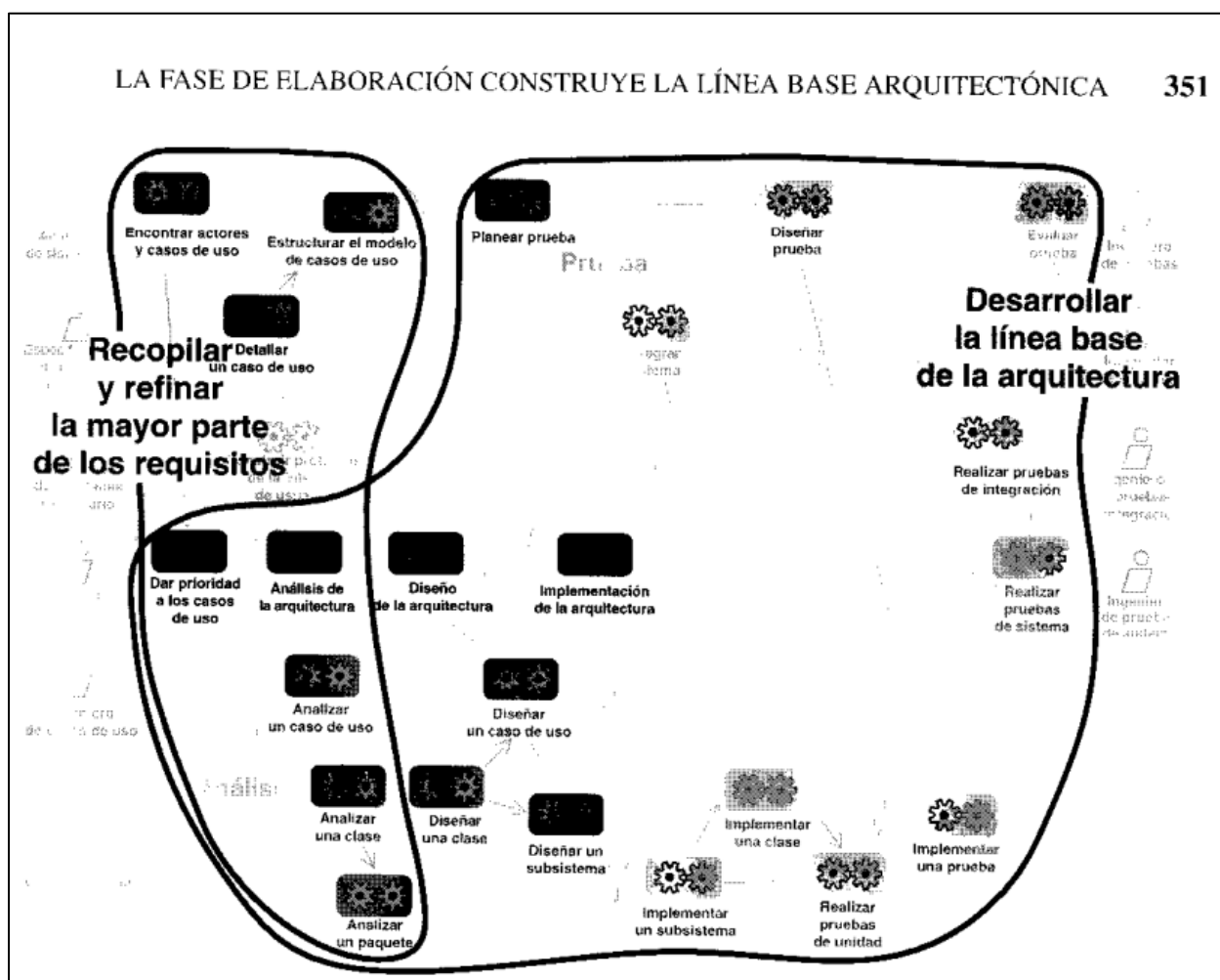


Figura 8. Fase de Elaboración

1.4.1.2.3. Construcción

La fase de construcción representa el momento en que el proyecto materializa el diseño arquitectónico en un producto funcional. Como señalan Jacobson, Booch y Rumbaugh (2000), "el

propósito primordial de esta fase es dejar listo un producto software en su versión operativa inicial, a veces llamada 'versión beta'" (p. 367).

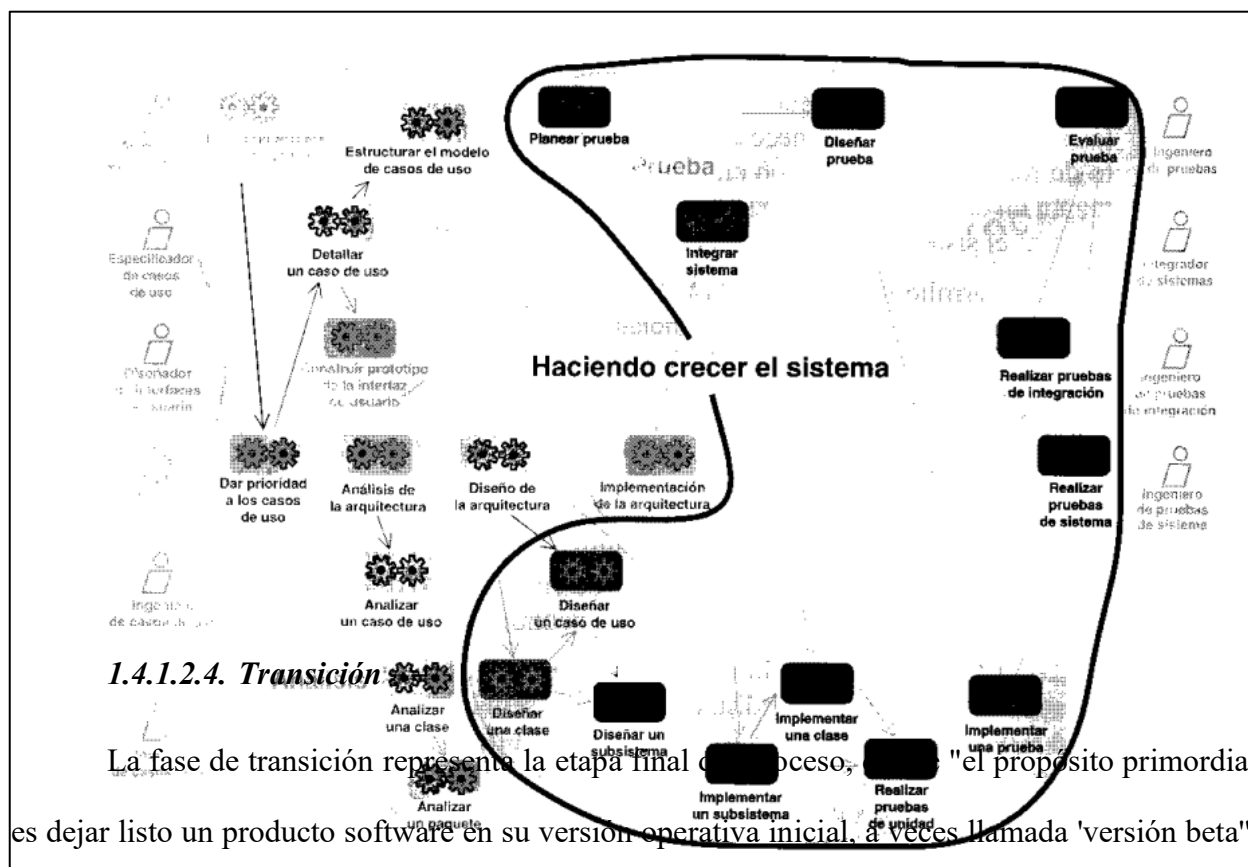
Durante esta fase se produce un cambio significativo de enfoque en el proyecto. "Mientras que las fases de inicio y elaboración podrían ser consideradas como investigación, la fase de construcción es análoga al desarrollo" (p. 368). El equipo parte de la línea base arquitectónica establecida en la elaboración para incrementalmente construir el sistema completo.

El proyecto aborda esta fase a través de múltiples iteraciones, cada una centrada en implementar un conjunto específico de casos de uso. Como explican los autores, "un desarrollo iterativo, guiado por los casos de uso y centrado en la arquitectura, construye el software mediante pequeños incrementos, y añade cada incremento a la acumulación previa de incrementos de tal forma que siempre se tenga una construcción ejecutable" (p. 371). Esta aproximación incremental permite gestionar mejor la complejidad y "aísla en gran parte riesgos y defectos dentro del ámbito de una construcción individual, haciendo que sean más fáciles de encontrar y tratar" (p. 371).

A lo largo de esta fase, los cinco flujos de trabajo operan con diferentes intensidades. En las primeras iteraciones, el énfasis se concentra en completar los requisitos pendientes (aproximadamente el 20% restante) y el análisis. Progresivamente, el foco se desplaza hacia el diseño, la implementación y las pruebas, que constituyen aproximadamente el 90% del esfuerzo total de esta fase.

El flujo de trabajo de implementación cobra especial relevancia, ya que "es en este flujo de trabajo en el que el proyecto lleva a cabo la mayor parte del trabajo de la fase de construcción, construyendo los componentes" (p. 375). El responsable de la integración del sistema establece un plan que define la secuencia de construcciones, generalmente comenzando por las capas inferiores de la arquitectura y expandiéndose progresivamente hacia las capas superiores.

Al finalizar esta fase, el producto debe tener "la calidad adecuada para su aplicación y asegurarse de cumplir los requisitos" (p. 367), quedando listo para entrar en la fase de transición.



(Jacobson et al., 2000, p. 367). Al entrar en esta fase, "el sistema ha alcanzado la capacidad operativa inicial" (p. 381), lo que significa que se considera suficientemente maduro para operar en el entorno del usuario.

Para el sistema de gestión de eventos universitarios basado en blockchain, los objetivos fundamentales de esta fase serán:

- "Cumplir los requisitos, establecidos en las fases anteriores, hasta la satisfacción de todos los usuarios" (p. 381), asegurando que la plataforma satisfaga plenamente las necesidades de la DTIC y de los diferentes tipos de usuarios universitarios.
- "Gestionar todos los aspectos relativos a la operación en el entorno del usuario, incluyendo la corrección de los defectos remitidos por los usuarios de la versión beta o por los encargados de las pruebas de aceptación" (p. 381).

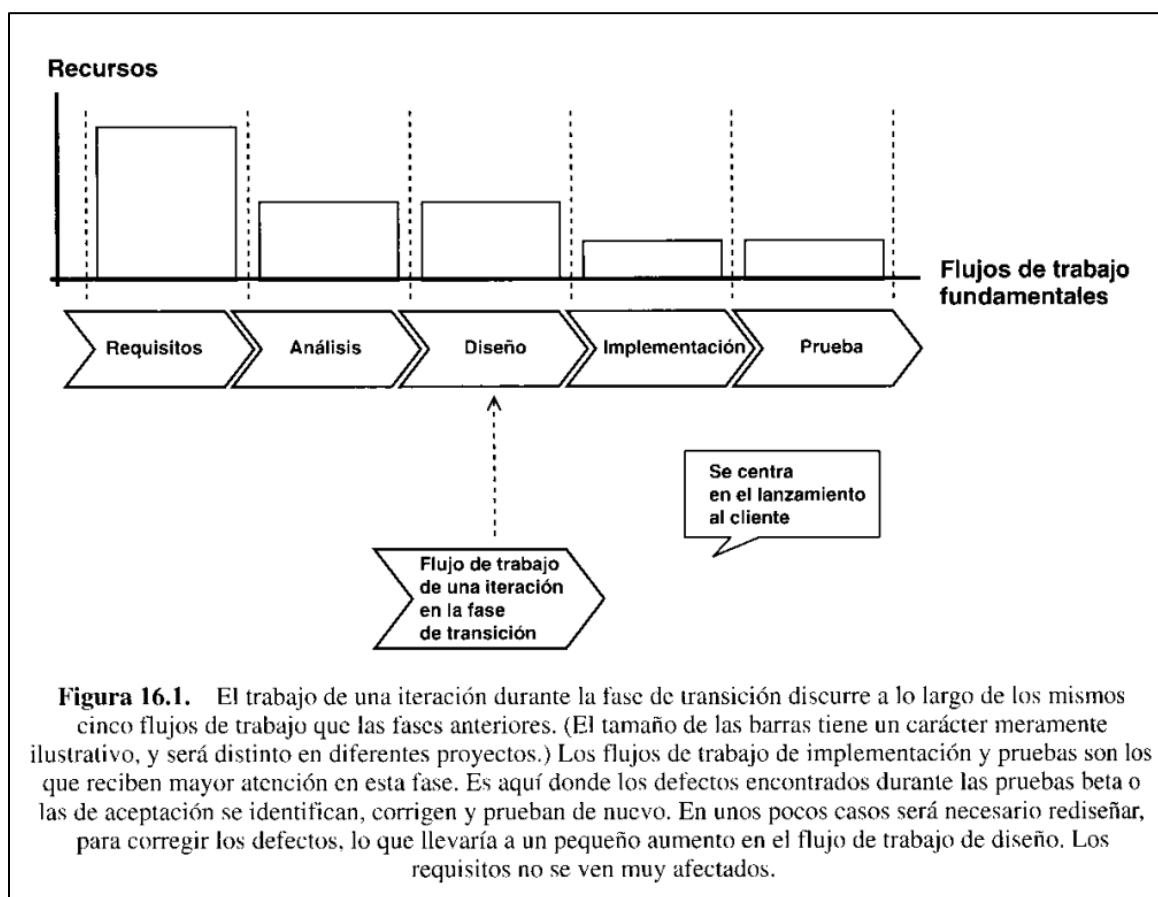
Durante esta fase, la actividad principal se concentrará en la implementación de correcciones y mejoras menores identificadas durante la operación en entorno real. El proyecto seguirá un flujo de trabajo que incluye:

1. "Preparar la versión beta (o de pruebas de aceptación) a partir de la versión con capacidad operativa inicial" (p. 386), seleccionando los eventos universitarios iniciales donde se implementará.
2. "Instalar (o preparar la instalación de) esta versión en los lugares elegidos" (p. 386), lo que puede implicar la migración de datos desde sistemas anteriores de gestión de eventos.
3. "Actuar a partir de la información recogida en las instalaciones de pruebas" (p. 386), realizando los ajustes necesarios.
4. "Adaptar el producto corregido a las circunstancias de los usuarios" (p. 386).

Como señalan los autores, "en esta fase no se busca reformular el producto" (p. 382), sino realizar ajustes menores que puedan implementarse dentro de la arquitectura establecida.

Cualquier cambio significativo identificado durante esta fase deberá postergarse para una versión posterior.

La fase concluye cuando se alcanza la satisfacción del cliente, que en el contexto universitario puede determinarse cuando "el sistema pase las pruebas de aceptación" (p. 390) establecidas por la DTIC, y demuestre su funcionamiento efectivo en eventos reales.



1.4.2. Complementarios de la Metodología

1.4.2.1. UML

El Lenguaje Unificado de Modelado (UML, por sus siglas en inglés) constituye un elemento fundamental dentro del Proceso Unificado de Desarrollo de Software, funcionando como

el lenguaje estándar para visualizar, especificar, construir y documentar los artefactos de un sistema. La versión 2.5 del UML, publicada en marzo de 2015 por el Object Management Group (OMG), representa una evolución significativa que simplifica y consolida las especificaciones anteriores.

UML proporciona un conjunto de notaciones gráficas estandarizadas que permiten representar de manera clara y precisa tanto aspectos estáticos como dinámicos de un sistema software. Su función principal en el contexto del PUDS es facilitar la comunicación entre todos los participantes del proyecto, desde analistas y desarrolladores hasta usuarios finales y stakeholders.

El lenguaje está organizado en dos grandes categorías de diagramas:

1. Diagramas Estructurales: Representan los elementos que componen un sistema y sus relaciones.
 - Diagrama de Clases: Describe la estructura estática del sistema mostrando las clases, sus atributos, operaciones y relaciones entre ellas. En el proyecto de blockchain, permite modelar entidades como Usuario, Evento, Entrada y sus interrelaciones.
 - Diagrama de Componentes: Muestra la organización y dependencias entre componentes físicos del software, útil para visualizar la arquitectura técnica del sistema.
 - Diagrama de Despliegue: Representa la disposición física de los artefactos en los nodos de hardware, crucial para definir la infraestructura que soportará la solución blockchain.
2. Diagramas de Comportamiento: Capturan los aspectos dinámicos del sistema.

- Diagrama de Casos de Uso: Visualiza la interacción entre el sistema y sus actores externos, representando los requisitos funcionales desde la perspectiva del usuario.
- Diagrama de Secuencia: Muestra la interacción entre objetos a través del tiempo, especialmente útil para representar flujos como la verificación biométrica o la emisión de tokens.
- Diagrama de Estados: Describe los diferentes estados por los que puede pasar un objeto a lo largo de su ciclo de vida, como el estado de una entrada (disponible, reservada, verificada, utilizada).
- Diagrama de Actividades: Representa flujos de trabajo paso a paso, modelando procesos como el registro para un evento o la validación de acceso.

Dentro del PUDS, UML no es simplemente una herramienta de documentación, sino un mecanismo fundamental para soportar el desarrollo iterativo e incremental. Los diagramas evolucionan a lo largo de las diferentes fases:

- En la fase de Inicio, los diagramas de casos de uso ayudan a definir el alcance del sistema.
- Durante la Elaboración, los diagramas estructurales permiten definir la arquitectura base.
- En la Construcción, los diagramas se refinan y detallan para guiar la implementación.
- Durante la Transición, los diagramas se actualizan para reflejar el sistema final entregado.

1.4.2.2. IEEE 830

El estándar IEEE 830-1998 proporciona un marco formal para la Especificación de Requisitos de Software (SRS, por sus siglas en inglés), constituyendo un complemento metodológico fundamental para el Proceso Unificado de Desarrollo de Software, especialmente durante la fase de captura de requisitos.

Este estándar define directrices para la estructura y contenido de las especificaciones de requisitos, teniendo como objetivo principal asegurar que estos sean completos, consistentes, inequívocos, verificables, modificables y trazables. Para el sistema de gestión de eventos universitarios con blockchain, la aplicación de este estándar permitirá documentar formalmente los requisitos del producto, sirviendo como base contractual entre el equipo de desarrollo y la DTIC.

La estructura recomendada por el IEEE 830-1998 comprende las siguientes secciones principales:

1. Introducción

- Propósito del documento
- Ámbito del sistema
- Definiciones, acrónimos y abreviaturas
- Referencias
- Visión general del documento

2. Descripción General

- Perspectiva del producto
- Funciones del producto
- Características de los usuarios
- Restricciones generales
- Suposiciones y dependencias

3. Requisitos Específicos

- Requisitos funcionales
- Requisitos no funcionales
- Requisitos de interfaz
- Requisitos de rendimiento
- Atributos del sistema
- Otros requisitos

En el contexto del proyecto de blockchain, esta estructura facilitará la documentación detallada de aspectos críticos como:

- Los requisitos de seguridad asociados a la emisión y validación de tickets digitales
- Las interfaces necesarias entre los sistemas blockchain y los sistemas universitarios existentes
- Los parámetros de rendimiento para operaciones concurrentes en eventos masivos
- Los requisitos relacionados con la verificación biométrica de identidad

El IEEE 830-1998 se complementa naturalmente con UML y el PUDS, sirviendo como un estándar para la documentación formal de los requisitos identificados durante la fase de inicio y refinados en la fase de elaboración. Mientras los casos de uso proporcionan una vista orientada al comportamiento desde la perspectiva del usuario, la especificación de requisitos según IEEE 830 ofrece una documentación estructurada y exhaustiva de todos los aspectos del sistema.

Cabe mencionar que, aunque este estándar fue reemplazado posteriormente por el IEEE 29148-2011, que integra varios estándares relacionados con ingeniería de requisitos, el IEEE 830-1998 sigue siendo ampliamente reconocido y utilizado como referencia base para la documentación de requisitos en muchas organizaciones.

1.5. Alcance y Limitaciones

1.5.1. Funciones Principales

1.5.1.1. Módulo de Autenticación

Gestiona la autenticación y autorización de usuarios en el sistema. Implementa estrategias de seguridad como JWT (JSON Web Tokens), manejo de sesiones, y protección de rutas según roles. Permite el registro de usuarios, inicio de sesión, y recuperación de contraseñas.

1.5.1.2. Módulo de Eventos

Núcleo del sistema para la creación y gestión de eventos universitarios. Permite a los organizadores definir eventos con múltiples parámetros: nombre, descripción, lugar, horarios, capacidad, categorías, reglas de acceso y tipos de entrada disponibles.

1.5.1.3. Módulo de Identidad

Implementa la verificación de identidad mediante IA. Procesa documentos de identificación y selfies para validar la identidad de usuarios cuando realizan compras masivas, aplicando algoritmos de reconocimiento facial y detección de documentos falsos.

1.5.1.4. Módulo de Inquilinos

Permite la segmentación del sistema para diferentes unidades universitarias (facultades, departamentos, etc.), garantizando que cada entidad pueda gestionar sus propios eventos de forma independiente pero dentro de una plataforma unificada.

1.5.1.5. Módulo de Blockchain

Gestiona la interacción con la tecnología blockchain. Encargado de la emisión de tokens únicos para cada entrada, garantizando su inmutabilidad, trazabilidad y resistencia a la falsificación mediante contratos inteligentes.

1.5.1.6. Módulo de Pagos

Administra todo el proceso de transacciones económicas en el sistema. Integra diferentes métodos de pago, procesa transacciones, emite comprobantes y gestiona devoluciones cuando sea necesario.

1.5.1.7. Módulo de Semillas

Proporciona datos iniciales y de prueba para el sistema. Facilita la configuración de entornos de desarrollo y demostración con eventos preconfigurados, usuarios de prueba y otros datos necesarios.

1.5.1.8. Módulo de Usuarios

Gestiona los perfiles de usuario, incluyendo información personal, preferencias, historial de eventos y entradas adquiridas. Implementa diferentes niveles de acceso según el tipo de usuario (administrador, organizador, asistente).

1.5.1.9. Módulo de Entradas y Compras

Estos módulos gestionan todo el ciclo de vida de los boletos electrónicos:

- **Módulo de Entradas:** Administra la creación, emisión y validación de entradas individuales, incluyendo la generación de códigos QR y su vinculación con tokens blockchain.
- **Módulo de Compras de Entradas:** Controla el proceso de adquisición de entradas, desde la selección hasta el pago, incluyendo reglas de negocio como límites de compra o descuentos por categoría de usuario.

1.5.1.10. Módulo de Auditoría

Registra todas las operaciones críticas del sistema para garantizar la trazabilidad y cumplimiento normativo. Almacena logs detallados de acciones como creación de eventos, emisión de entradas, pagos procesados y validaciones de acceso.

1.5.1.11. Módulo de Almacenamiento en la Nube

Gestiona el almacenamiento y procesamiento de archivos multimedia en la plataforma. Permite subir, optimizar y servir imágenes para eventos, perfiles de usuario y documentos de identidad de forma segura y eficiente.

1.5.1.12. Módulo Común

Proporciona funcionalidades y utilidades compartidas entre los diferentes módulos del sistema. Incluye componentes reutilizables, servicios comunes, interceptores, filtros y pipes que mantienen la consistencia en toda la aplicación.

1.5.1.13. Módulo de Configuración

Gestiona las variables de entorno y configuraciones del sistema para diferentes entornos (desarrollo, pruebas, producción). Centraliza parámetros como claves API, conexiones a servicios externos y ajustes de comportamiento del sistema.

1.5.1.14. Módulo de Utilidades

Proporciona funciones auxiliares y herramientas para operaciones comunes como formateo de datos, validaciones, generación de identificadores únicos y otras operaciones de apoyo utilizadas por múltiples módulos del sistema.

1.5.2. Usuarios Beneficiados

El software está diseñado para cubrir las necesidades de distintos perfiles que interactúan con Evently:

- **Organizadores.** Usuarios con permisos para crear eventos, definir sectores, subir logos y personalizar sus tickets.
- **Asistentes.** Estudiantes o usuarios que navegan por los eventos, compran entradas y validan su QR el día del evento.
- **Administradores Universitarios:** Unidad central DICIT que supervisa, aprueba el uso del sistema y accede a reportes generales por facultad.

1.5.3. Limitaciones y Restricciones

1.5.3.1. Limitaciones

1.5.3.1.1. Limitaciones técnicas

Compatibilidad con dispositivos. La verificación biométrica y el escaneo QR tienen requisitos mínimos de hardware que podrían excluir a usuarios con dispositivos móviles antiguos o de gama baja, especialmente en una comunidad universitaria diversa como la de la UAGRM.

1.5.3.1.2. Limitaciones operativas

Cobertura de eventos. El sistema está optimizado para eventos académicos, culturales y deportivos típicos del entorno universitario de la UAGRM. No está diseñado para gestionar eventos externos de gran escala o con requisitos especializados fuera del ámbito académico.

Personalización por facultad. Aunque cada facultad puede configurar sus eventos, existen límites en el nivel de personalización visual y funcional que pueden implementar, debiendo adherirse a los parámetros generales establecidos por la DTIC para mantener la coherencia del sistema.

1.5.3.1.3. Limitaciones de alcance

Gestión financiera avanzada. Aunque permite procesar pagos básicos, el sistema no incluye funcionalidades avanzadas de contabilidad o gestión financiera integral que podrían requerir las unidades organizadoras para eventos complejos.

Verificación académica. El sistema verifica identidad, pero no valida automáticamente elegibilidad académica (como requisitos de matrícula o pertenencia a programas específicos) para eventos que requieran dichas validaciones.

1.5.3.2. Restricciones

1.5.3.2.1. Restricciones técnicas

Dependencia de Servicios de IA. La validación de identidad mediante reconocimiento facial y documental depende de servicios externos como AWS Rekognition, Google Vision o Azure Face. La calidad del resultado puede verse afectada por fotos borrosas, mala iluminación o dispositivos sin cámara, lo que podría limitar la validación automática y requerir revisión manual.

Operaciones Blockchain. El tiempo de respuesta al generar una entrada única (hash + QR) dependerá de la latencia de la red blockchain utilizada. El uso de soluciones L2(zk-sync o pruebas de conocimiento 0) como Sepolia Testnets se considera para minimizar costos y tiempos, pero sigue existiendo una ligera demora respecto a una base de datos tradicional.

Almacenamiento Multimedia Externo. Fotos de documentos y selfies requieren un almacenamiento seguro en la nube (ej. Cloudinary, Pinata), que también tiene limitaciones como políticas de acceso, capacidad gratuita y costos variables según uso.

Escaneo en Tiempo Real. La lectura de códigos QR debe ser rápida y confiable en el día del evento. Esto dependerá del dispositivo del escáner y de la conectividad del entorno, por lo que el sistema debe optimizar la lectura offline o con reconexión automática.

1.5.3.2.2. Restricciones legales

Las facultades, centros o unidades académicas que utilicen el sistema para organizar eventos serán responsables del uso adecuado de los datos que soliciten a los usuarios, especialmente en procesos como verificación de identidad mediante documento y fotografía. En caso de solicitar documentación adicional (como cédula de identidad o datos biométricos), el

organizador deberá justificar su uso, resguardar dicha información conforme a la normativa vigente, y asumir las consecuencias legales en caso de mal manejo o filtración de información.

El sistema deberá cumplir estrictamente con las disposiciones establecidas en la **Ley N° 164 de Telecomunicaciones, Tecnologías de Información y Comunicación**, particularmente en su Título VII sobre los derechos a la privacidad y protección de datos personales. Esta normativa establece que toda persona tiene derecho a que su información personal sea tratada con confidencialidad, requerirá consentimiento informado y deberá ser utilizada exclusivamente para los fines autorizados. En el contexto del presente sistema, esto incluye la recopilación y uso de datos como nombre, correo electrónico, documento de identidad, imágenes de verificación facial y cualquier otro dato asociado a la compra y validación de entradas para eventos universitarios.

1.5.3.2.3. Restricciones de recursos

Costos de Servicios Externos. El uso de servicios como AWS Rekognition, Cloudinary, servidores Node.js y nodos blockchain implica un costo recurrente proporcional al volumen de uso. El sistema deberá tener una estrategia de recuperación de esos costos (ej. comisión por entrada vendida).

Requisitos Técnicos. El sistema debe desplegarse en infraestructura que soporte tráfico simultáneo, escaneo de QR en tiempo real y acceso desde múltiples dispositivos. Requiere buen ancho de banda, balanceadores de carga y almacenamiento en la nube.

Recursos Humanos. El equipo de desarrollo debe tener experiencia en backend escalable, servicios en la nube (cloud), blockchain y consumo de APIs de IA. También es necesario contar con alguien que administre seguridad y bases de datos en producción.

1.6. Recursos Necesarios

1.6.1. Software y Herramientas

El desarrollo del sistema requerirá una combinación de tecnologías modernas, multiplataforma y de código abierto para garantizar escalabilidad y seguridad:

1.6.1.1. Lenguajes y Frameworks Principales

- TypeScript: Utilizado como lenguaje principal tanto en frontend como backend
- Angular: Framework frontend (versión 15 o superior)
- NestJS: Framework backend, seleccionado por su estructura modular y compatibilidad con arquitecturas limpias
- Solidity: Lenguaje para desarrollo de contratos inteligentes en blockchain

1.6.1.2. Tecnologías Blockchain

- Hardhat: Entorno de desarrollo para compilar, desplegar y probar contratos inteligentes
- Sepolia Testnet: Red de pruebas Ethereum para desarrollo antes de producción

1.6.1.3. Almacenamiento de Datos

- PostgreSQL: Sistema de gestión de bases de datos relacional, robusto y con soporte para tipos de datos JSON
- Cloudinary: Para almacenamiento multimedia, especialmente imágenes de verificación

1.6.1.4. Herramientas de Desarrollo y Despliegue

- Visual Studio Code: IDE principal con extensiones para TypeScript, Angular y Solidity
- Docker: Para contenerización de servicios y consistencia entre entornos
- Git y GitHub: Control de versiones y colaboración
- Insomnia: Herramienta para pruebas y documentación de API REST
- Jira: Plataforma para gestión ágil del proyecto, seguimiento de tareas y sprints

1.6.1.5. Servicios y Utilidades

- AWS Rekognition: Para verificación de identidad mediante reconocimiento facial
- Cohere: Para validation de identidad,y evitar compras masivas de entradas mediante el envío de prompts.
- Pinata: Para subir imágenes descentralizadas prioritarias como lo que viene siendo lo qr de entradas.

Esta combinación de tecnologías permite construir un sistema robusto que integra verificación de identidad, emisión de entradas tokenizadas y gestión centralizada de eventos universitarios para la UAGRM, utilizando herramientas modernas y de código abierto.

1.6.2. Equipos y Hardware

1.6.2.1. Equipos de Desarrollo

- Estaciones de trabajo: Computadoras con mínimo 8 GB de RAM (16 GB recomendados para desarrollo blockchain), procesador i5 o equivalente, SSD de 256 GB o superior

- Requisitos específicos: Capacidad suficiente para ejecutar Visual Studio Code, entornos Docker, y realizar pruebas de integración y despliegue de contratos inteligentes a través de Hardhat
- Conexión a internet: Estable y de alta velocidad para interacción con testnets de Ethereum y servicios en la nube

1.6.2.2. Infraestructura de Servidor

- Servidores de aplicación: Instancias AWS EC2 (mínimo t3.medium) para el despliegue de servicios de backend NestJS
- Servidores de base de datos: Instancia AWS RDS para PostgreSQL con alta disponibilidad
- Almacenamiento: Servicios S3 para archivos estáticos y copias de seguridad

1.6.2.3. Dispositivos para Validación y Producción

- Lectores de códigos QR: Dispositivos móviles o lectores especializados con cámara de alta resolución para validación de entradas en eventos
- Teléfonos móviles: Dispositivos Android e iOS de diferentes características para pruebas de compatibilidad y experiencia de usuario
- Tablets: Para uso de organizadores de eventos durante registro y control de acceso
- Equipos de escritorio: Para pruebas de compatibilidad en distintos navegadores (Chrome, Firefox, Safari, Edge)

1.6.2.4. Hardware Complementario

- Routers y equipos de red: Para simulación de entornos con diferentes condiciones de conectividad

- Monitores secundarios: Para facilitar tareas de desarrollo y diseño
- Sistemas de respaldo energético: UPS para servidores locales de desarrollo que eviten pérdidas por cortes eléctricos
- Dispositivos de almacenamiento: Discos externos para copias de seguridad locales

1.6.3. Presupuesto Estimado

El proyecto será desarrollado bajo una estrategia de optimización de recursos, aprovechando la infraestructura existente de la UAGRM y priorizando tecnologías open source. Se considera el siguiente desglose aproximado:

Recursos	Fecha		Cantidad	Costo Unitario	% Depreciacion	Costo Total
	Desde	Hasta				
Hardware						
Laptop rtx 3050	31/3/2025	20/6/2025	2	1319	25	1979
Monitores	31/3/2025	20/6/2025	4	200	20	640
Software						
AWS EC2	31/3/2025	20/6/2025	1	1979,4	0	1979,4
Architect Enterprise Pro	31/3/2025	20/6/2025	5	245	0	1225
Windows 11	31/3/2025	20/6/2025	5	145	0	725
Jira Enterprise	31/3/2025	20/6/2025	1	77,4	0	77,4
Pinata Picnic	31/3/2025	20/6/2025	4	20	0	80
Cohere Command R+	31/3/2025	20/6/2025	1	12		12
Cloudinary Plus	31/3/2025	20/6/2025	4	89	0	356
Postman Basic	31/3/2025	20/6/2025	1	380	0	380

GitHub Pro	31/3/2025	20/6/2025	5	16	0	80
Gente						
Desarrollador	31/3/2025	20/6/2025	4	4000	0	16000
Gestor de proyecto	31/3/2025	20/6/2025	1	4800	0	4800
Infraestructura						
Espacio de oficina	31/3/2025	20/6/2025	1	3200	0	3200
Servicio de Agua	31/3/2025	20/6/2025	1	36		144
Servicio de Internet	31/3/2025	20/6/2025	1	104	0	104
Servicio de Electricidad	31/3/2025	20/6/2025	1	176	0	176
Logística						
Capacitación Ethereum	31/3/2025	20/6/2025	1	60	0	60
Capacitación AWS	31/3/2025	20/6/2025	1	60	0	60
Transporte	31/3/2025	20/6/2025	80 días	3	0	1200
Alimentación	31/3/2025	20/6/2025	80 días	4	0	1600
Total						34,877.8

1.7. Cronograma

1.7.1. Plan de Ejecución

El desarrollo del sistema de gestión de eventos universitarios con blockchain seguirá un enfoque iterativo e incremental basado en el Proceso Unificado de Desarrollo de Software (PUDS). Esta metodología permitirá la construcción progresiva del producto, con validaciones constantes

de cada componente y la posibilidad de refinar los requisitos según los resultados obtenidos en cada fase.

El cronograma ha sido estructurado considerando los recursos tecnológicos disponibles en la DTIC de la UAGRM, la complejidad inherente a la integración de tecnologías blockchain con sistemas convencionales, y la priorización de módulos según su valor para los usuarios. Cada ciclo de desarrollo se orientará a entregar funcionalidades mínimas viables que puedan ser evaluadas por representantes de los tres grupos de usuarios (organizadores, asistentes y administradores universitarios).

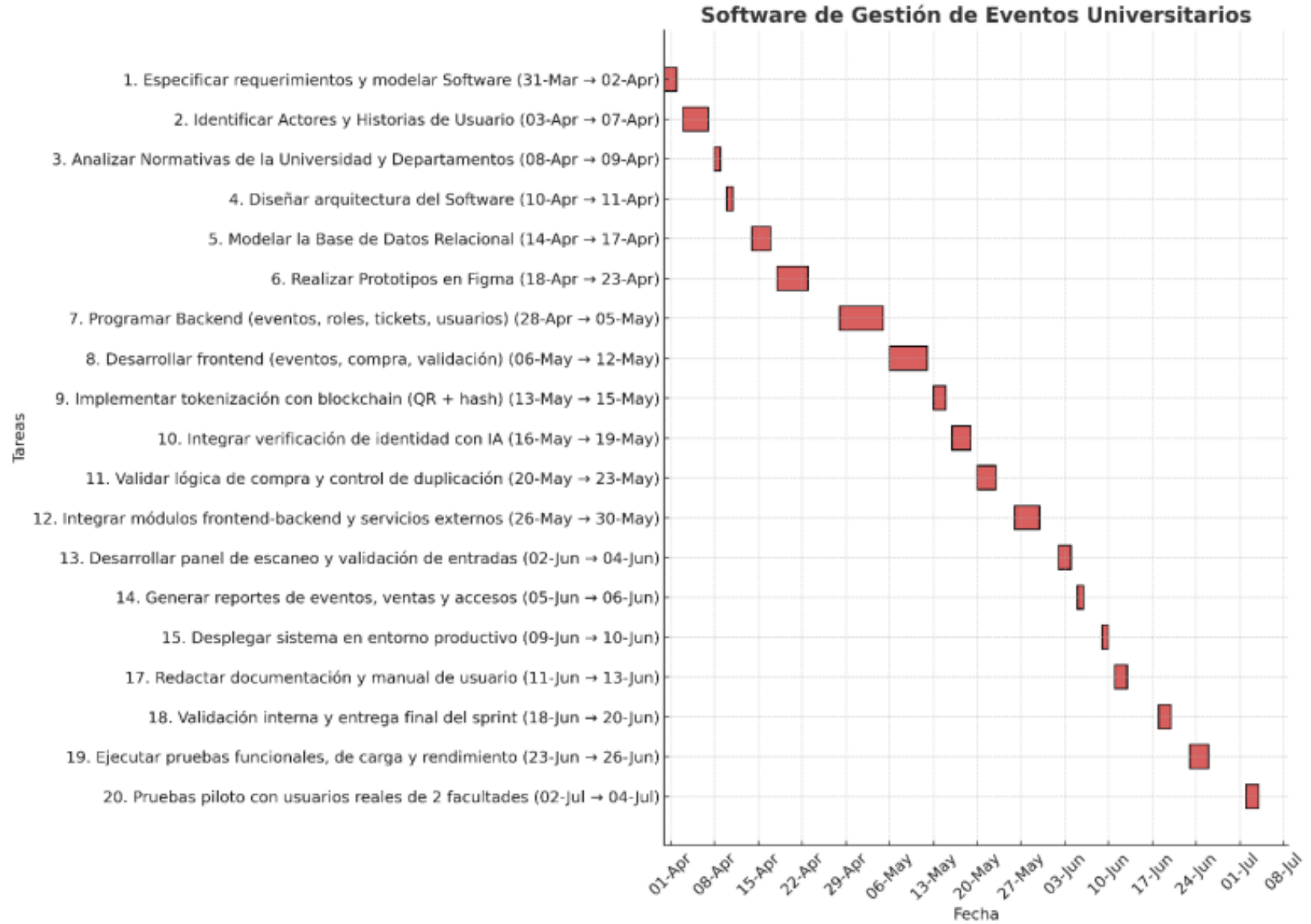


Ilustración 1. Diagrama de Gantt

1.7.2. Hitos importantes

Durante el ciclo de desarrollo del sistema de gestión de eventos universitarios con blockchain, se identifican los siguientes hitos clave que marcan la evolución del proyecto desde su inicio hasta la implementación:

- Hito 1 – Inicio y especificación de requerimientos: Formalización del proyecto, modelado inicial del software y definición de alcance. *31 de marzo de 2025*
- Hito 2 – Diseño de arquitectura del sistema: Entrega del modelo de dominio, historias de usuario, arquitectura técnica y diseño de la base de datos relacional. *17 de abril de 2025*
- Hito 3 – Primer prototipo funcional: Finalización de prototipos en Figma y validación del diseño de experiencia de usuario con stakeholders. *23 de abril de 2025*
- Hito 4 – Desarrollo backend completo: Implementación de módulos core para gestión de eventos, roles, tickets y usuarios en NestJS. *5 de mayo de 2025*
- Hito 5 – Desarrollo frontend principal: Interfaces de usuario para navegación de eventos, proceso de compra y validación de entradas. *12 de mayo de 2025*
- Hito 6 – Integración blockchain: Implementación del sistema de tokenización con contratos inteligentes para emisión de entradas seguras. *15 de mayo de 2025*
- Hito 7 – Sistema de verificación con IA: Integración del módulo de verificación de identidad basado en reconocimiento facial. *19 de mayo de 2025*
- Hito 8 – Validación e integración completa: Sistema integrado con todos los módulos funcionando en conjunto, frontend-backend-blockchain. *30 de mayo de 2025*
- Hito 9 – Despliegue en producción: Implementación del sistema en los servidores de la D TIC y configuración final. *10 de junio de 2025*

Capítulo 2 Blockchain Fundamentos Teóricos para la Gestión Descentralizada de Eventos Universitarios

2.1. Blockchain

Blockchain o cadena de bloques es una tecnología de registro distribuido que almacena información en bloques enlazados y asegurados mediante criptografía. Según Nakamoto (2008), el blockchain fue inicialmente concebido como la tecnología subyacente de Bitcoin, diseñada para resolver el problema del doble gasto en monedas digitales sin necesidad de una autoridad central.

Tapscott y Tapscott (2016) describen esta tecnología como un sistema que permite a partes que no confían entre sí compartir datos digitales con total seguridad a través de un libro mayor distribuido, eliminando intermediarios y proporcionando mayor transparencia e inmutabilidad en los registros.

Estructura y funcionamiento

La unidad fundamental del blockchain es el bloque, que representa una colección de datos que registra todas las transacciones o información generada en un período específico de tiempo. Cada bloque está identificado por una altura de bloque única que aumenta secuencialmente con cada nuevo bloque generado, funcionando como una "unidad de tiempo" en el ecosistema blockchain (CESS, 2023).

Un bloque típicamente consta de dos componentes principales:

- **Encabezado:** Contiene metadatos críticos como la altura del bloque, el hash del bloque anterior (hash de padres), la raíz de transacción, la raíz del estado y el digesto.
- **Cuerpo:** Contiene el grupo de transacciones validadas durante ese período.

El término "blockchain" hace referencia precisamente a esta cadena de bloques donde cada uno está conectado criptográficamente con el anterior, formando una secuencia inmutable de

información (*Ilustración 2*). Una vez que un bloque es validado y añadido a la cadena, se sincroniza con todos los nodos de la red, haciendo que la información contenida no pueda ser eliminada ni manipulada, garantizando así la confiabilidad e integridad de los datos.

Aplicación específica en el proyecto

En el presente proyecto, la tecnología blockchain cumple un papel fundamental en la gestión de eventos universitarios para la D.T.I.C. de la Universidad Autónoma Gabriel René Moreno, con las siguientes aplicaciones específicas:

- **Tokenización de entradas:** Cada entrada a un evento se representa como un token único en la blockchain, garantizando su autenticidad y evitando la duplicación o falsificación. Estos tokens contienen información encriptada que identifica al evento, ubicación, asiento y propietario legítimo.
- **Trazabilidad completa:** La blockchain registra inmutablemente cada transacción relacionada con los tokens de entrada, desde su emisión inicial hasta cualquier transferencia posterior, proporcionando un historial completo y transparente de la propiedad.
- **Transferencia segura:** La arquitectura descentralizada de blockchain permite que los estudiantes y personal universitario puedan transferir o revender sus entradas de manera segura, sin posibilidad de fraude, ya que cada transacción es verificada por la red antes de ser confirmada.
- **Validación de acceso:** El sistema utiliza los tokens almacenados en la blockchain para verificar la legitimidad de las entradas mediante códigos QR, complementando esta verificación con reconocimiento facial para garantizar que solo el propietario legítimo pueda acceder al evento.

- Auditoría y transparencia: La naturaleza inmutable de blockchain proporciona a los administradores universitarios un registro permanente y verificable de todas las operaciones relacionadas con los eventos, facilitando la auditoría y el análisis posterior.

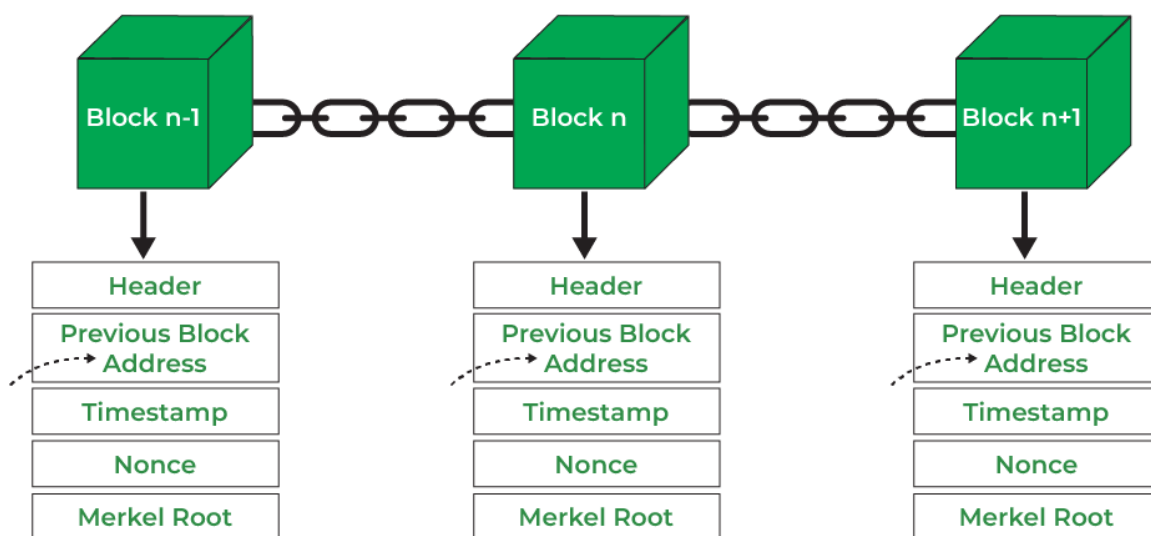


Ilustración 2. Estructura básica de una cadena de bloques

Nota. En esta figura podemos apreciar cómo cada bloque contiene datos, un hash propio y el hash del bloque anterior, creando una cadena criptográficamente asegurada. Los cambios en cualquier bloque alterarían todos los bloques subsiguientes, haciendo evidente cualquier manipulación.

2.2. Ethereum

2.2.1. Red Ethereum

Es una plataforma computacional descentralizada basada en blockchain que permite la creación y ejecución de contratos inteligentes. Buterin (2014, p. 14) define Ethereum de manera más precisa como:

"Una plataforma Turing-completa de procesamiento y ejecución de contratos basado en un libro contable de cadena de bloques. No es un clon de Bitcoin, sino un diseño e implementación

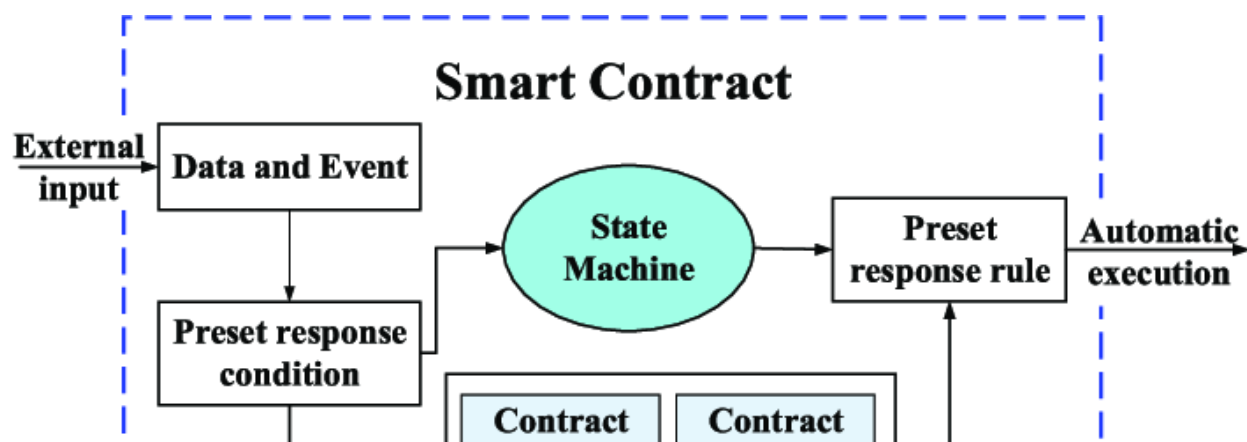
completamente independiente. Ethereum tiene una moneda incorporada, llamada ether, que se requiere para pagar por la ejecución de contratos. La cadena de bloques de Ethereum registra los contratos, que se expresan en un nivel bajo, en un lenguaje Turing-completo, similar a un código de bytes. En esencia, un contrato es un programa que se ejecuta en cada nodo del sistema Ethereum. Los contratos Ethereum pueden almacenar datos, enviar y recibir pagos de ether, almacenar ether, y ejecutar una gama infinita (de ahí Turing-completo) de las acciones computables, actuando como agentes de software autónomos descentralizados."

Como señala Buterin (2014, p. 14), "Ethereum puede implementar sistemas bastante complejos que en otro caso se implementarían como cadenas alternativas", lo que demuestra su versatilidad para crear aplicaciones descentralizadas de diversa complejidad, como sistemas de registro de nombres similares a Namecoin, pero implementados directamente sobre la infraestructura de Ethereum.

2.2.2. Contratos Inteligentes

Los contratos inteligentes son programas autónomos que se ejecutan exactamente como fueron programados, sin posibilidad de censura, inactividad, fraude o interferencia de terceros. Szabo (1996) los definió inicialmente como "un conjunto de promesas, especificadas en formato digital, incluidos los protocolos dentro de los cuales las partes cumplen estas promesas".

En el contexto de Ethereum, Wood (2014) describe los contratos inteligentes como entidades autónomas que residen en la blockchain Ethereum, con su propio código y almacenamiento persistente. Estos contratos se activan mediante transacciones y pueden interactuar con otros contratos, tomar decisiones, almacenar datos y enviar ether.



2.2.3. Estructura de una Transacción en Ethereum

Las transacciones en Ethereum no son simplemente transferencias de valor, sino elementos complejos que pueden desencadenar cambios en el estado global de la red. Según la Ethereum Foundation (2020), una transacción es un mensaje firmado criptográficamente que incluye la dirección del remitente, la dirección del destinatario, la cantidad de ether a transferir, los datos de entrada para un contrato inteligente, los límites de gas, el precio del gas y un número único para prevenir ataques de repetición.

Como explica Wood (2014), "cada transacción contiene una carga útil de datos arbitrarios y una serie de otros parámetros, pero la característica más importante es que debe ser validada por cada nodo participante" (p. 5).

En la ilustración 4, se puede ver una representación gráfica de una transacción en Ethereum. La estructura muestra cómo cada transacción incluye no solo información sobre el remitente y el destinatario, sino también parámetros específicos que determinan cómo se ejecutará y el costo computacional asociado.

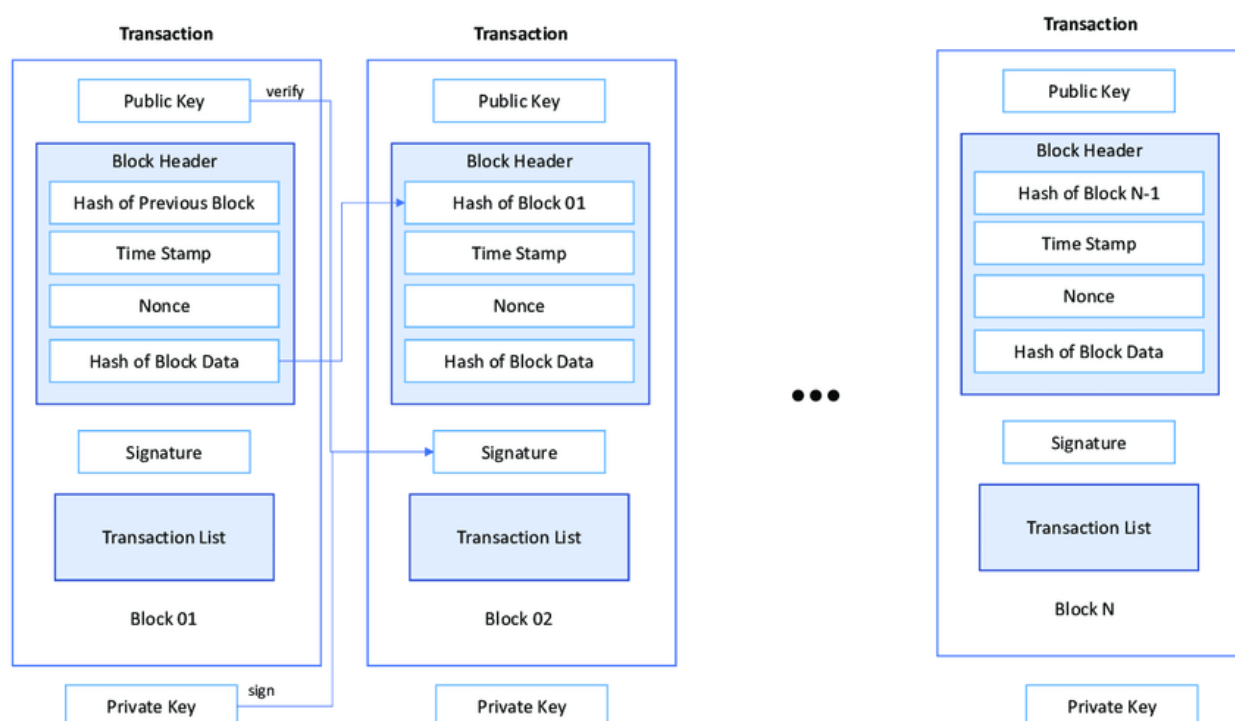


Ilustración 4. Estructura de una Transacción en Ethereum

2.2.4. Gas y Mecanismos de Consenso

El gas es la unidad que mide el trabajo computacional requerido para ejecutar operaciones en la red Ethereum. Buterin (2014) afirma que el gas es esencial para prevenir el abuso de la red, ya que cada operación tiene un costo proporcional a los recursos que consume. Este sistema permite que la red mantenga su integridad frente a posibles ataques de denegación de servicio.

El mecanismo de consenso es el protocolo que garantiza que todos los nodos de la red lleguen a un acuerdo sobre el estado actual de la blockchain. Ethereum originalmente utilizaba

Proof of Work (PoW), pero ha migrado a Proof of Stake (PoS) con Ethereum 2.0, reduciendo significativamente su consumo energético y mejorando la escalabilidad (Ethereum Foundation, 2022).

2.3. Soluciones de Capa 2 y Zero-Knowledge Proofs

2.3.1. zkSync

zkSync es una solución de escalabilidad de capa 2 para Ethereum que utiliza pruebas de conocimiento cero para aumentar el rendimiento de las transacciones. Según Matter Labs (2020), zkSync permite hasta 2,000 transacciones por segundo (TPS) mientras mantiene la seguridad de Ethereum, comparado con las aproximadamente 15 TPS de la capa base de Ethereum.

2.3.2. Zero-Knowledge Proofs

Una prueba de conocimiento cero (ZK proof) es un método criptográfico que permite a una parte (el probador) demostrar a otra (el verificador) que conoce un valor o cumple una condición, sin revelar ninguna información adicional. Como indican Goldwasser, Micali y Rackoff (1989), "estas pruebas son zero-knowledge porque no revelan ninguna información más allá de la validez de la afirmación siendo probada".

Ben-Sasson et al. (2018) explican que las ZK-SNARKs (Zero-Knowledge Succinct Non-Interactive Argument of Knowledge) son un tipo específico de prueba de conocimiento cero que permite verificaciones eficientes sin interacción entre probador y verificador, siendo fundamentales para la privacidad y escalabilidad en blockchain.

2.3.3. Red Sepolia

Sepolia es una red de prueba (testnet) para Ethereum, diseñada para proporcionar un entorno seguro donde los desarrolladores pueden probar sus aplicaciones descentralizadas antes de implementarlas en la red principal. Según la Ethereum Foundation (2020), Sepolia utiliza el

mismo mecanismo de consenso que la red principal, pero con ether sin valor real, permitiendo experimentar sin riesgos financieros.

2.3.4. Aplicaciones Descentralizadas (dApps)

Las aplicaciones descentralizadas o dApps son aplicaciones que se ejecutan en una red blockchain en lugar de en servidores centralizados. McConaghy et al. (2016) definen las dApps como "aplicaciones de software que se ejecutan en redes peer-to-peer de computadoras en lugar de en una sola computadora, poniendo a los usuarios en control de sus datos y sin un punto único de fallo".

2.3.5. Tokenización

La tokenización es el proceso de convertir derechos sobre un activo en una representación digital en blockchain. Tapscott y Tapscott (2016) describen la tokenización como "la creación de tokens digitales que representan activos del mundo real, permitiendo su intercambio, programación y transferencia de valor de manera instantánea y segura". Este concepto es fundamental para la representación de eventos universitarios, certificaciones y credenciales académicas en un sistema blockchain.

2.4. Otras tecnologías complementarias

Además de las tecnologías principales, el sistema de gestión de eventos universitarios incorpora una serie de herramientas y frameworks complementarios que fortalecen su arquitectura, seguridad y experiencia de usuario. Estas tecnologías han sido seleccionadas por su madurez, soporte en la comunidad y compatibilidad con los objetivos del proyecto. A continuación, se detallan sus características, beneficios y roles dentro de la solución propuesta:

Frontend: Angular

Angular es un framework de desarrollo web mantenido por Google, basado en TypeScript, que permite construir aplicaciones de una sola página (SPA) con arquitectura modular, componentes reutilizables y alto rendimiento.

- Se utiliza para desarrollar tanto la interfaz pública de eventos como el panel administrativo para organizadores.
- Proporciona una experiencia fluida y reactiva gracias al uso de data binding bidireccional, lazy loading y rutas protegidas.
- Facilita la creación de formularios reactivos para el registro de usuarios, compra de entradas y verificación de identidad.
- Permite la implementación de interfaces adaptables que funcionan correctamente en dispositivos móviles y de escritorio.
- Su sistema de módulos permite escalar la aplicación manteniendo un código organizado y mantenible.

Esta tecnología resulta ideal para el desarrollo de aplicaciones web progresivas (PWA) que permitan a los usuarios acceder a sus entradas incluso sin conexión constante a internet.



Figura 10. Framework Angular

Backend: NestJS

NestJS es un framework progresivo de Node.js que facilita la construcción de aplicaciones del lado del servidor utilizando TypeScript. Está inspirado en Angular y ofrece una arquitectura modular y escalable.

- Proporciona una estructura organizada mediante decoradores, controladores y proveedores de servicios.
- Facilita la implementación de guardias de autenticación y autorización para proteger rutas y recursos.
- Ofrece integración nativa con TypeORM para la comunicación con PostgreSQL.
- Permite la implementación de middlewares para validación, logging y transformación de datos.
- Soporta la creación de servicios de WebSockets para actualizaciones en tiempo real durante eventos.
- Facilita la implementación de interceptores para manipular respuestas y solicitudes HTTP.

NestJS garantiza una arquitectura de backend robusta que permite escalar el sistema conforme aumente el número de eventos y usuarios registrados en la plataforma.



Base de Datos: PostgreSQL

PostgreSQL es un sistema de gestión de bases de datos relacional de código abierto, reconocido por su fiabilidad, características avanzadas y cumplimiento con estándares SQL.

- Ofrece soporte para tipos de datos avanzados como JSON, arrays y datos geoespaciales útiles para la gestión de eventos.
- Proporciona características de integridad referencial para garantizar consistencia en los datos relacionados con eventos, entradas y usuarios.
- Incluye funcionalidades de alta disponibilidad y respaldo automático para evitar pérdida de datos.
- Permite la implementación de procedimientos almacenados y triggers para validaciones complejas.
- Ofrece extensiones como pgcrypto para el almacenamiento seguro de información sensible.
- Soporta índices especializados que optimizan consultas frecuentes sobre eventos y transacciones.

La robustez de PostgreSQL es fundamental para mantener la integridad de los datos en un sistema donde las transacciones de compra y validación de entradas requieren consistencia y seguridad.



Figura 12. SGBD PostgreSQL

Blockchain: Hardhat

Hardhat es un entorno de desarrollo para Ethereum que facilita la compilación, prueba y despliegue de contratos inteligentes escritos en Solidity, proporcionando herramientas para agilizar el proceso de desarrollo blockchain.

- Facilita la creación y prueba de contratos inteligentes para la tokenización de entradas a eventos.
- Proporciona una red local para pruebas sin costos de gas durante el desarrollo.
- Incluye herramientas de depuración que permiten rastrear errores en transacciones y contratos.
- Permite la integración con billeteras web como MetaMask para pruebas de usuario realistas.
- Soporta la ejecución de tareas automatizadas para despliegue y verificación de contratos.
- Facilita la interacción entre las APIs de NestJS y los contratos desplegados en la blockchain.

Hardhat constituye un componente esencial para la implementación segura de la verificación de entradas mediante tokens no fungibles (NFTs), garantizando la autenticidad y evitando la falsificación o duplicación de entradas a eventos universitarios.



Figura 13. Framework Hardhat JS

Capítulo 3 D.T.I.C.

3.1. La Empresa

3.1.1. Nombre

La empresa se denomina "D.T.I.C." (Dirección de Tecnologías de Información y Comunicación), departamento tecnológico de la Universidad Autónoma Gabriel René Moreno dedicado a la gestión e implementación de soluciones informáticas para los diversos procesos académicos y administrativos de la institución.

3.1.2. Misión

Proporcionar soluciones tecnológicas innovadoras y servicios informáticos de calidad que impulsen la transformación digital de la Universidad Autónoma Gabriel René Moreno, garantizando el funcionamiento óptimo de la infraestructura tecnológica, la seguridad de la información y el soporte técnico eficiente para fortalecer los procesos académicos, de investigación y administrativos de la institución.

3.1.3. Visión

Consolidarse como un departamento tecnológico de referencia entre las universidades bolivianas, liderando la implementación de tecnologías emergentes como blockchain, inteligencia artificial y computación en la nube, para posicionar a la UAGRM a la vanguardia de la innovación digital, fomentando un ecosistema académico conectado, seguro y tecnológicamente avanzado que responda a los desafíos de la educación superior del siglo XXI.

3.1.4. Imagen Corporativa



Figura 14. Logo de la Empresa

3.1.5. Web Contacto

La comunicación oficial se realiza a través de su sitio web (<https://www.uagrm.edu.bo>) y facilitando el acceso a información y servicios para toda la comunidad.



3.2. Estructura Organizacional

EMPRESA

ORGANIGRAMA DE LA

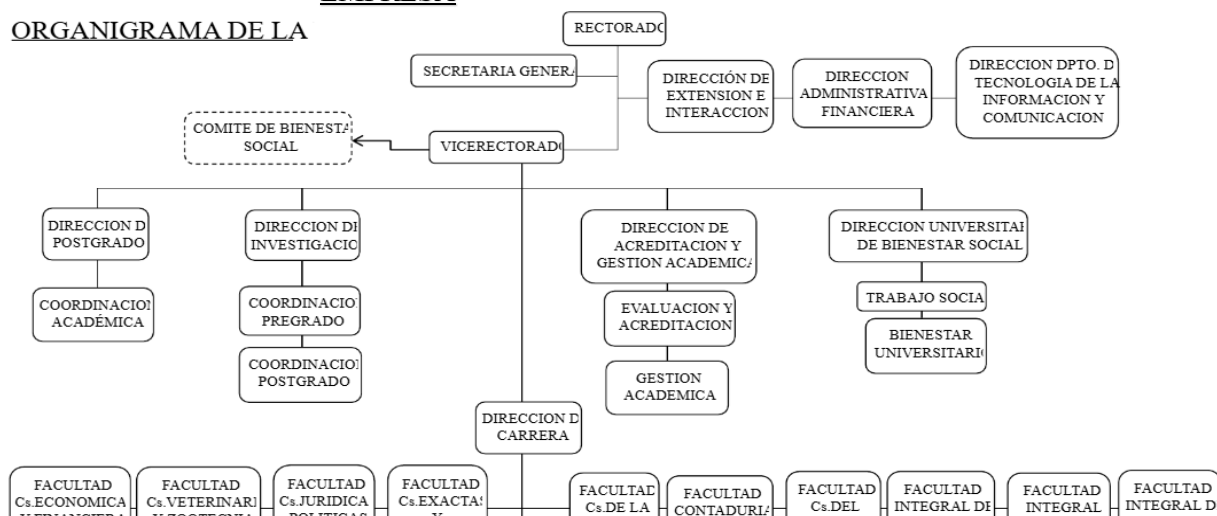
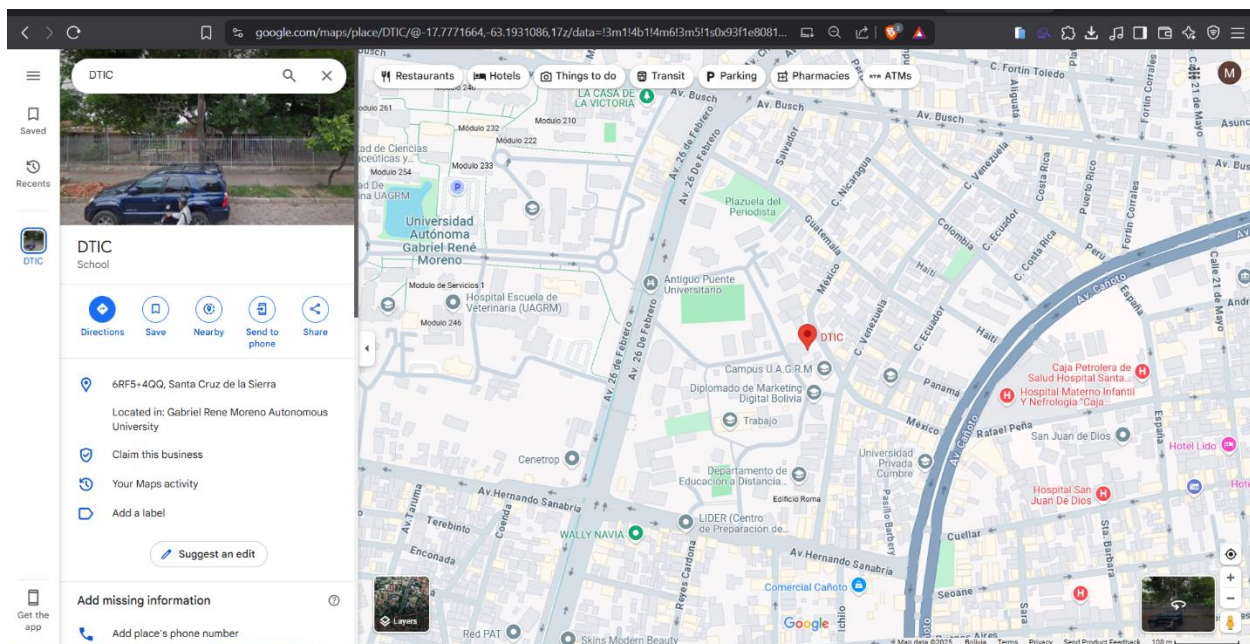


Figura 16. Estructura organizacional de la empresa

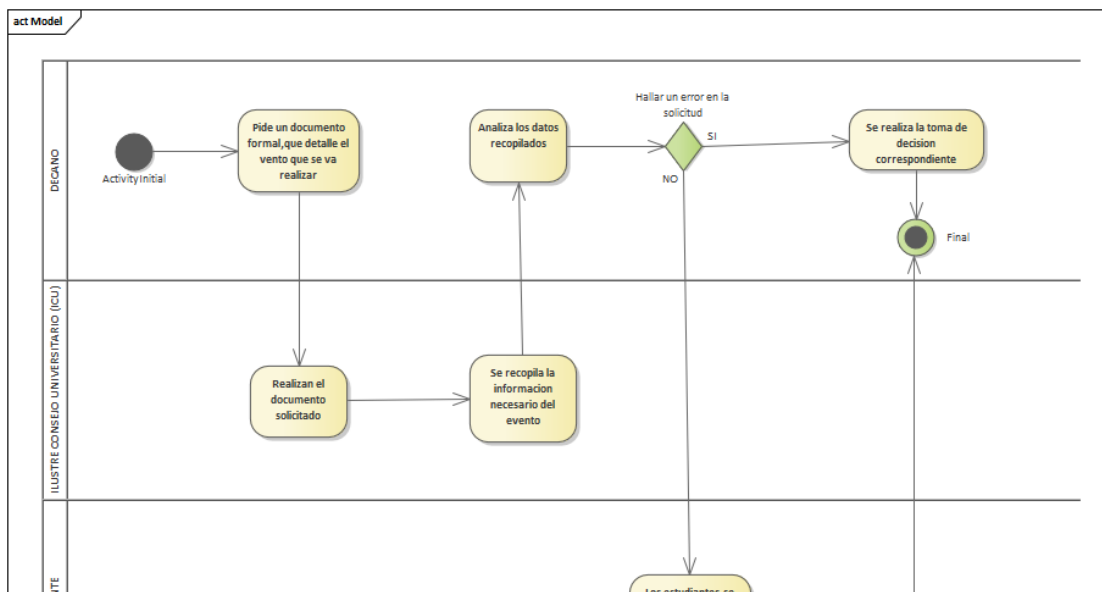
3.3. Ubicación e Infraestructura



3.4. Modelo de Negocio

Figura 17. Ubicación

3.4.1. Flujograma para la publicación de un evento



Capítulo 4 Especificación de Requerimientos de Software (E.R.S)

4.1. Introducción

En este documento se definirán las necesidades del cliente en términos de requisitos; se especificarán y detallarán los requisitos funcionales y no funcionales. Esta especificación se ha estructurado basándose en las directrices dadas por el estándar IEEE830 para desarrollo del presente proyecto.

4.1.1. Propósito

El presente documento tiene como objetivo definir de manera clara y verificable los requisitos del "Software de Gestión de Eventos Universitarios con Blockchain" para la Universidad Autónoma Gabriel René Moreno. Se especificarán las funcionalidades esenciales y las restricciones que deberá cumplir el sistema, facilitando la comunicación entre desarrolladores y organizadores.

4.1.2. *Ámbito del sistema*

Nombre del Proyecto: Sistema de Gestión de Entradas y Eventos Universitarios

Funcionalidades que va a realizar:

- Gestión de eventos universitarios, permitiendo la creación, configuración y publicación de eventos académicos y culturales.
- Emisión y comercialización de entradas digitales mediante la integración con tecnología blockchain para garantizar su autenticidad y trazabilidad.
- Verificación y validación de entradas en tiempo real a través de códigos QR y autenticación en la blockchain.
- Registro de asistentes y generación de listados de participantes por evento.
- Creación de reportes detallados con estadísticas de ventas, accesos y participación en formatos PDF y XLSX.
- Gestión de usuarios con roles y permisos diferenciados para organizadores, verificadores y administradores.
- Notificaciones automáticas para recordatorios de eventos y confirmación de entradas.

Beneficios que va a dar el proyecto:

- Automatiza y moderniza la gestión de eventos universitarios, reduciendo errores y tiempos de procesamiento.
- Aumenta la seguridad de las transacciones eliminando la posibilidad de falsificación o duplicación de entradas.
- Facilita la organización y análisis de eventos con reportes centralizados y herramientas estadísticas.

- Ofrece una experiencia moderna y digital para los asistentes, con entradas accesibles directamente desde sus dispositivos móviles.
- Mejora la transparencia de las operaciones mediante el uso de blockchain, permitiendo auditar la trazabilidad de cada transacción.
- Incrementa la eficiencia organizativa al centralizar la gestión de eventos en una única plataforma tecnológica.

4.1.3. Definiciones acrónimos y observaciones

Tabla 1. Definiciones de acrónimos y observaciones

Nombre	Descripción
RF	Requerimiento funcional
RNF	Requerimiento no funcional
ERS	Especificación de requerimientos del software
QR	Código de respuesta rápida utilizado para la validación de entradas.
JS	Lenguaje de programación para desarrollar el software
Blockchain	Tecnología que permite registrar transacciones de forma inmutable y segura.
Hardhat	Herramienta para el desarrollo y pruebas de contratos inteligentes en blockchain.

API	Interfaz de Programación de Aplicaciones
UI/UX	Interfaz de Usuario y Experiencia de Usuario
NESTJS	Framework backend para Node.js con arquitectura modular y TypeScript.
AWS	Amazon Web Services, plataforma de servicios en la nube

4.1.4. Referencias

Especificación de requisitos según el estándar de IEEE 830-1998 (IEEE, 2008)

4.1.5. Visión general del documento

El presente documento consta de tres secciones. En la primera sección se ofrece una introducción que establece el propósito, el alcance, las definiciones, las referencias y la estructura global de la Especificación de Requisitos de Software (ERS). En la segunda sección se describe el software de manera general, resaltando sus principales funciones, restricciones y dependencias, lo que permite comprender el contexto y la operatividad del sistema sin entrar en detalles técnicos profundos. Finalmente, la tercera sección define de forma detallada los requisitos que debe cumplir el sistema, abarcando aspectos funcionales y no funcionales, interfaces, estructura de la base de datos y otros criterios esenciales para su desarrollo.

4.2. Descripción general

4.2.1. Perspectiva del proyecto

El **Sistema de Gestión de Entradas y Eventos Universitarios con Blockchain** se ofrece como una solución **SaaS (Software as a Service)** gestionada centralmente por la DTIC de la UAGRM. A continuación se describen sus relaciones con otros sistemas, su arquitectura general y su grado de independencia:

1. Relación con sistemas existentes

- a. **Portal de Servicios UAGRM:** punto único de acceso mediante OAuth2; el sistema hereda la autenticación institucional y los roles de usuario registrados en el SIE (Sistema de Información Estudiantil) .
- b. **Pasarela de pagos (Stripe):** gestiona las transacciones de suscripción de DTIC y los pagos de entradas; expone webhooks para notificar renovaciones y vencimientos de membresía.
- c. **Nodo Privado Hyperledger Fabric:** ejecuta los smart contracts para emisión y validación de tickets; se comunica vía SDK Go sobre gRPC para almacenar hashes Keccak256 de cada transacción.

2. Modelo de suscripciones y asignación de tenants

- a. Cuando una **facultad** desea organizar eventos, presenta una **solicitud formal** (carta o formulario interno) a la DTIC.
- b. Tras aprobar la solicitud, la **DTIC UAGRM** adquiere la membresía correspondiente (básica, media o premium), la cual define un **límite de eventos simultáneos**:
 - i. **Básica:** 1 evento.

- ii. **Media:** 3 eventos.
- iii. **Premium:** 5 eventos.
- c. Al activarse la suscripción, el sistema provisiona un **tenant lógico** para DTIC y crea automáticamente las **cuentas de superusuario** para DTIC y las cuentas de “organizador” para cada facultad solicitante.
- d. A partir de ese momento, las facultades utilizan sus cuentas de organizador para **crear y personalizar sus propios eventos** según las cuotas de su plan.

3. Interacción de roles

- a. **Superadministrador (DTIC):** gestiona solicitudes, suscripciones y tenants; asigna permisos de organizador y supervisa el uso de cuotas.
- b. **Organizadores (Facultades):** crean/administran eventos hasta el límite del plan contratado; definen precios, generan códigos QR y analizan métricas de asistencia.
- c. **Verificador de Acceso:** Personal designado por cada facultad para el control de puertas, escaneando los Qrs de los asistentes del evento. Consulta el estado de cada ticket (válido, usado o expirado) contra el smart contract en la blockchain.
- d. **Estudiantes / Público:** acceden al frontend web público para comprar entradas y validar QR en puerta; su validación consulta el smart contract en la blockchain.

4. Independencia vs. ecosistema mayor

A pesar de operar de forma autónoma (base de datos PostgreSQL y microservicios propios desplegados en contenedores Docker sobre

Kubernetes), el sistema forma parte del **ecosistema de servicios académicos** de la UAGRM, integrándose con la autenticación (Portal UAGRM), la facturación (Stripe) y el registro estudiantil (SIE) .

4.2.2. Funciones del producto

Las funciones del software son las siguientes:

- **F1. Gestión de Eventos**

- Los organizadores podrán crear eventos universitarios, configurando sus fechas, categorías de entradas y cantidad disponible para cada tipo.

- **F2. Venta y Preventa de Entradas**

- La plataforma debe habilitar a los usuarios la compra de entradas durante fases de preventa o venta, garantizando que no se exceda el límite de stock definido por los organizadores.

- **F3. Generación de Entradas con Identificadores Únicos**

- Cada entrada generada debe contar con un identificador único (hash keccak256) y código QR, que será registrado para su trazabilidad y posterior validación durante el acceso al evento.

- **F4. Control de Acceso mediante Validación QR**

- El sistema debe permitir validar la autenticidad de las entradas escaneando el QR el día del evento, asegurando que no se reutilicen o falsifiquen.
- **F5. Envío de notificación sobre la compra de la entrada**
 - La plataforma debe notificar al usuario sobre la compra de su entrada tras haber sido exitosa la compra.
- **F6. Monitoreo de Ventas, Auditoría y Detección de Compras Sospechosas**
 - El sistema debe proporcionar a los organizadores un panel de monitoreo en tiempo real con estadísticas de ventas, alertas de compras inusuales y reportes por facultad o sector.
- **F7. Verificación de Identidad en Compras Masivas con IA**
 - Cuando un usuario intente comprar más de un número límite de entradas (por ejemplo, 5), el sistema debe solicitar una verificación de identidad con documento y selfie, validando los datos mediante un sistema de inteligencia artificial.

4.2.3. Características de los usuarios

Tabla 2. Características de los usuarios

Actor	Rol	Características	Habilidades Técnicas	Restricciones
Super Administrador	DTIC	Personal de la Dirección de Tecnología e Información de la UAGRM, responsable de la gestión global del SaaS	Manejo de portal administrativo SaaS; gestión de suscripciones.	conexión a Internet estable; navegador moderno
Organizadores	Facultades	Personal administrativo de facultades, responsable de la creación y	Uso de la interfaz web de eventos; generación y configuración de códigos QR;	Cuenta de organizador activa; conexión a Internet; navegador moderno

		configuración de eventos.	análisis de reportes y métricas	
Verificador de Acceso	Control de puertas	Personal operativo encargado de validar el acceso físico al evento.	Uso de la aplicación móvil o web de validación de QR; manejo básico de dispositivo con cámara.	Dispositivo con cámara o lector QR; acceso al frontend de validación; conexión móvil o Wi-Fi
Estudiantes / Público	Asistentes	Estudiante y público general que adquiere entradas y asiste al evento	Uso de aplicaciones web o móviles básicas; lectura de instrucciones para despliegue de QR.	Cuenta institucional, dispositivo para mostrar QR y acceso a Internet

4.2.4. Restricciones

4.2.4.1 Requisitos de hardware

- Nodos de aplicación y blockchain: servidores con ≥ 8 GB RAM, 4 vCPU, SSD ≥ 100 GB.
- Dispositivos de validación: cámaras ≥ 8 MP y conexión Wi-Fi o 4G estable.

4.2.4.2 Requisitos de software y versiones

- **Blockchain**
 - Hyperledger Fabric ≥ 2.4
 - Hardhat ≥ 2.6
 - Nodo Ethereum (Sepolia) o RPC JSON-RPC
- **Capa de escalabilidad**
 - zkSync SDK ≥ 1.0
 - ZK-SNARKs (snarkjs ≥ 0.6)
- **Backend**
 - NestJS $\geq 9.x$ con Node.js $\geq 18.x$
 - TypeORM ≥ 0.3 para PostgreSQL ≥ 14
- **Frontend**
 - Angular ≥ 15 (PWA, ES2020)
 - ngx-qrcode2 ≥ 10.0

4.2.4.3 Limitaciones de red y gas fees

- Dependencia de latencias $RPC \leq 200$ ms.
- Gestión de monedero y fondos para gas en mainnet.

4.2.4.4 Entorno de desarrollo y despliegue

- **Contenerización:** toda la aplicación (backend, frontend y servicios de blockchain) se empaqueta en imágenes Docker (versión ≥ 20.10).
- **Despliegue:** las imágenes se ejecutan en instancias AWS EC2, configuradas con Amazon Linux 2 o Ubuntu LTS.
- **Automatización mínima:** se utiliza Docker Compose para orquestar los contenedores en EC2; opcionalmente, se pueden emplear scripts de AWS CLI para actualizar imágenes y reiniciar servicios.
- **Gestión de secretos:** las credenciales sensibles (API keys, private keys de smart contracts, credenciales de base de datos) se almacenan en AWS Systems Manager Parameter Store o AWS Secrets Manager, montadas como variables de entorno en los contenedores Docker.

4.2.4.5 Normativas y estándares

- ISO/IEC 27001, IEEE 830-1998, Ley 164 Bolivia, PCI DSS (si aplica).

4.2.4.6 Rendimiento y escalabilidad

- Tiempo de respuesta ≤ 2 s.
- ≥ 200 usuarios concurrentes sin degradación.
- Pruebas de hasta 2 000 TPS en módulo blockchain.

4.2.5. Suposiciones y dependencias

- Se asume que los usuarios contarán con dispositivos compatibles para leer códigos QR y acceso a una conexión de internet estable para interactuar con el sistema.
- Se presume que la red blockchain Sepolia, junto con las librerías criptográficas (por ejemplo, keccak256), funcionarán de manera estable y serán compatibles con la implementación prevista.
- Se asume que las facultades organizadoras de eventos tendrán personal capacitado o disponible para recibir formación sobre el uso del sistema.
- Se presume que los eventos iniciales no tendrán una concurrencia masiva que supere las 1,000 transacciones diarias durante los primeros seis meses de operación.
- Se asume que las entradas digitales serán aceptadas mayoritariamente por los asistentes como método principal de acceso a los eventos.

4.2.5.1. Dependencias

Dependencias de Software y Servicios

- El sistema dependerá de la red blockchain Sepolia para la emisión, validación y trazabilidad de las entradas.

- Se hará uso de servicios externos de verificación de identidad, como algoritmos de inteligencia artificial y reconocimiento facial, para garantizar la autenticidad de las transacciones.
- El sistema dependerá de PostgreSQL como base de datos principal para el almacenamiento de información relacionada con eventos, usuarios y transacciones.
- Se hará uso de servicios de almacenamiento en la nube (por ejemplo, AWS S3, Google Cloud Storage o Pinata) para alojar multimedia y datos asociados a los eventos.
- El sistema dependerá de APIs de terceros para la autenticación de usuarios mediante estándares como OAuth 2.0 (por ejemplo, Google Sign-In o similar).
- Se dependerá de servicios de envío de notificaciones por correo electrónico o mensajería instantánea (por ejemplo, SendGrid, Twilio o similares) para la comunicación con los usuarios.

Dependencias de Infraestructura

- El sistema requerirá servidores con capacidad suficiente para manejar tráfico simultáneo, con un mínimo de 8GB de RAM y procesadores de 4 núcleos.
- Se dependerá de una infraestructura de red estable y de alta disponibilidad para garantizar un tiempo de actividad superior al 99.9%.
- Se hará uso de balanceadores de carga y servicios de escalabilidad automática para soportar eventos de alta concurrencia.

Dependencias Legales y Regulatorias

- El sistema deberá cumplir con las normativas fiscales bolivianas, incluyendo la emisión de facturas electrónicas conforme a los requisitos del Servicio de Impuestos Nacionales (SIN).
- Se dependerá de la estabilidad de las normativas relacionadas con blockchain y comercio electrónico en Bolivia.
- El almacenamiento y manejo de datos personales deberán cumplir con las normativas locales de protección de datos y privacidad.

Dependencias Operativas

- Se dependerá de las facultades organizadoras para proporcionar información precisa y actualizada sobre los eventos a gestionar.
- La operación del sistema dependerá de la disponibilidad de personal técnico capacitado para su mantenimiento y actualización.

4.3. Requisitos específicos

En este capítulo se describen de manera precisa los requisitos que el sistema debe cumplir para satisfacer las necesidades de la DTIC de la UAGRM y de sus facultades usuarias.

Inicialmente se presentan los **requisitos funcionales** (las funciones y servicios que deberá ofrecer el software), seguidos de los **requisitos no funcionales** (criterios de calidad y restricciones de operación). A continuación se definen las **interfaces de usuario y de sistema**, los **requisitos de base de datos**, las **restricciones de diseño** y,

finalmente, los **requisitos de rendimiento**, estableciendo para cada uno los criterios de aceptación y las métricas asociadas.

Diagrama de Actividades

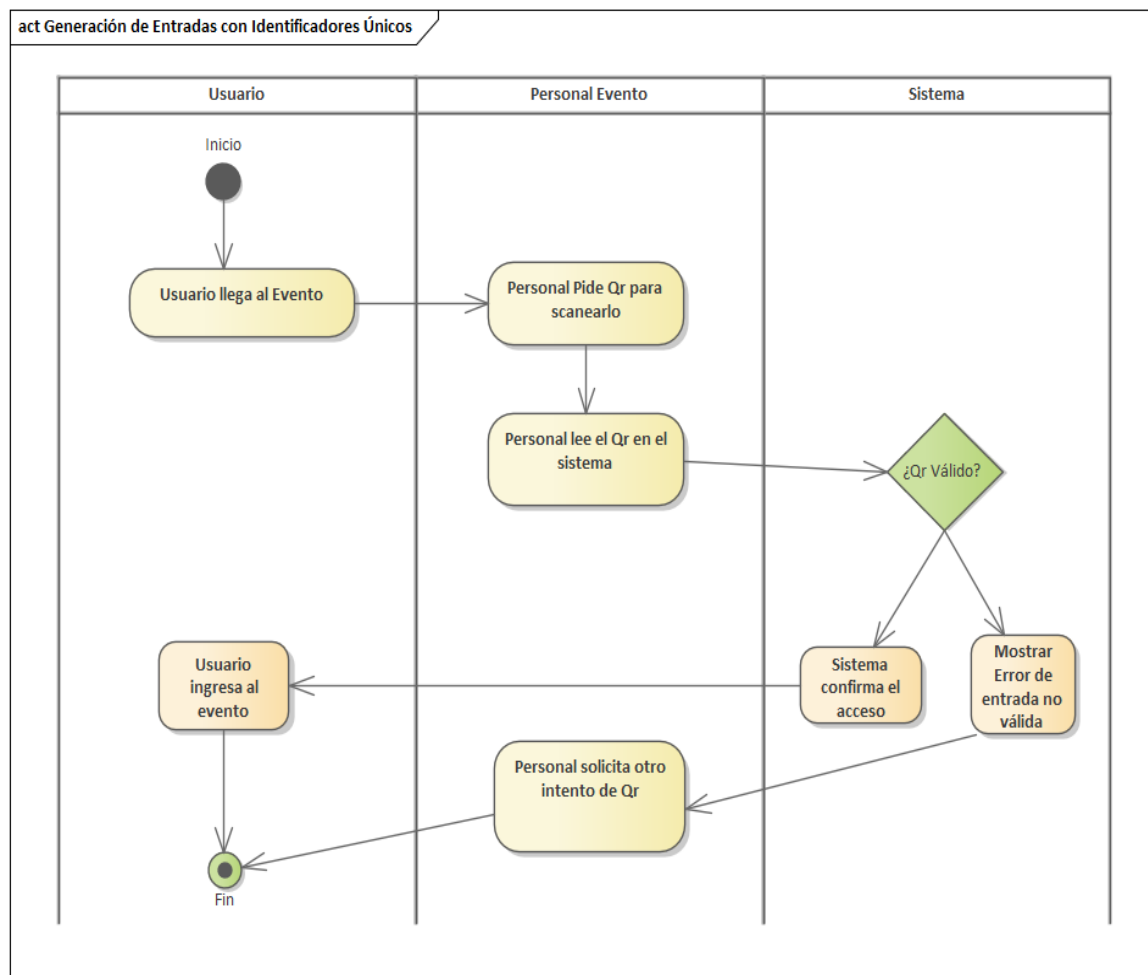


Figura 19. Generación de entradas con identificadores únicos

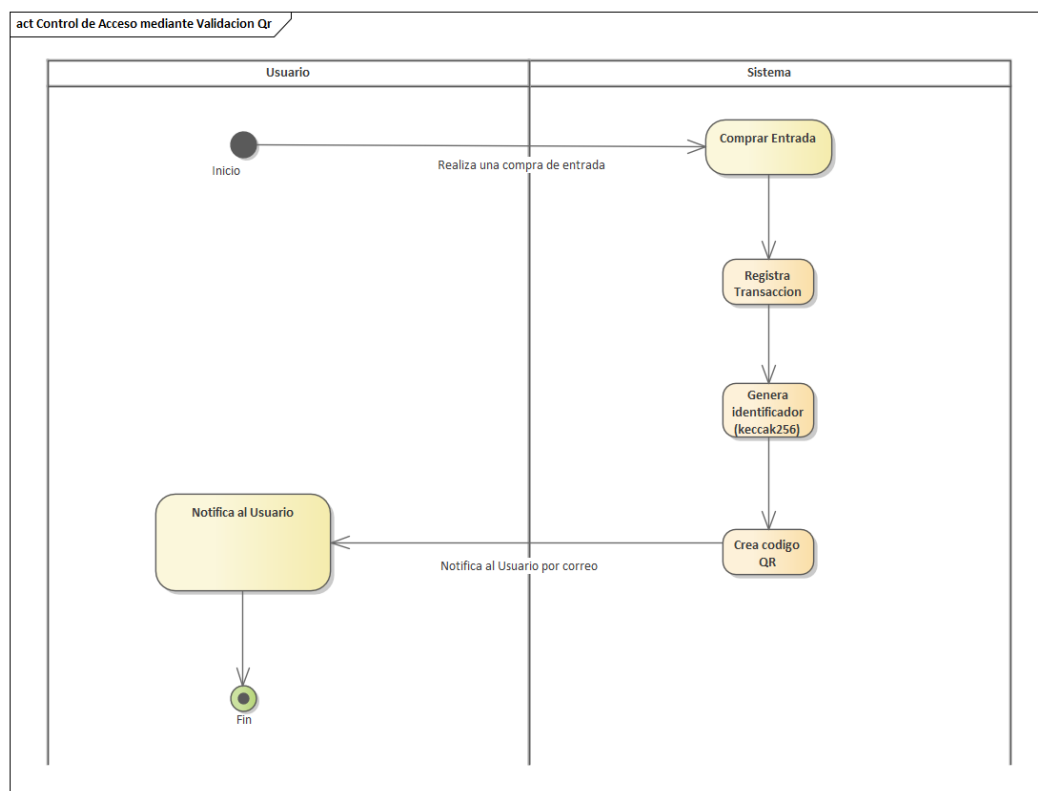


Figura 20. Control de acceso mediante validación QR

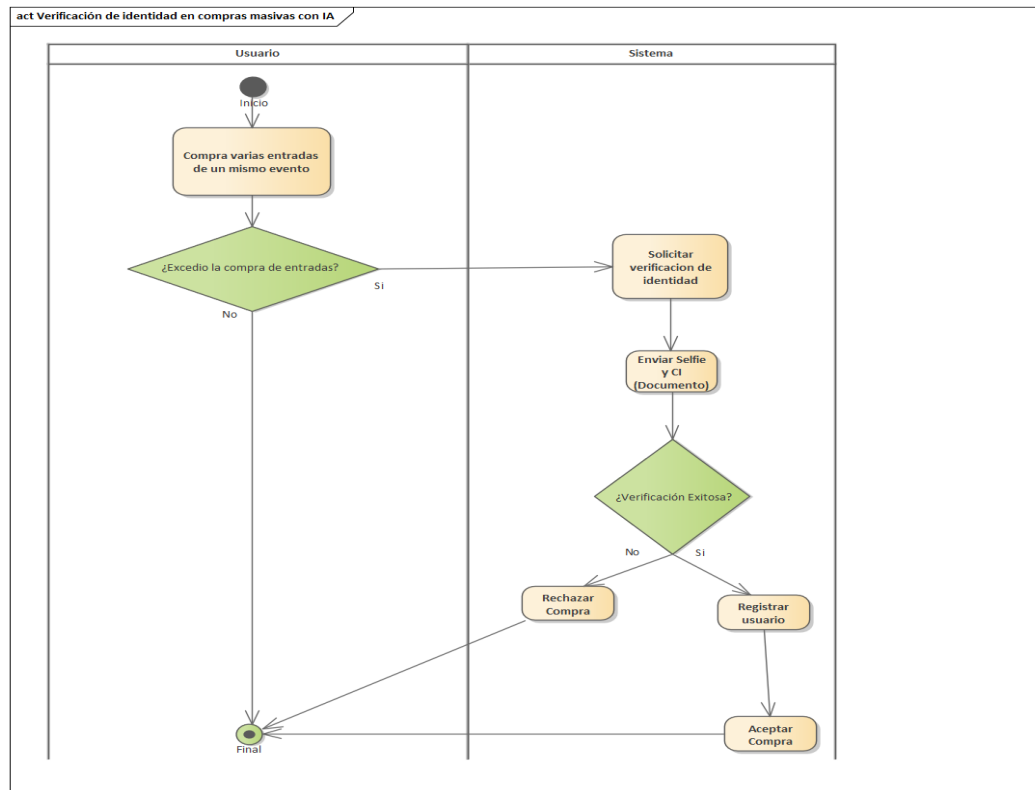


Figura 21. Verificación de identidad en compras masivas con IA

Modelo de Dominio

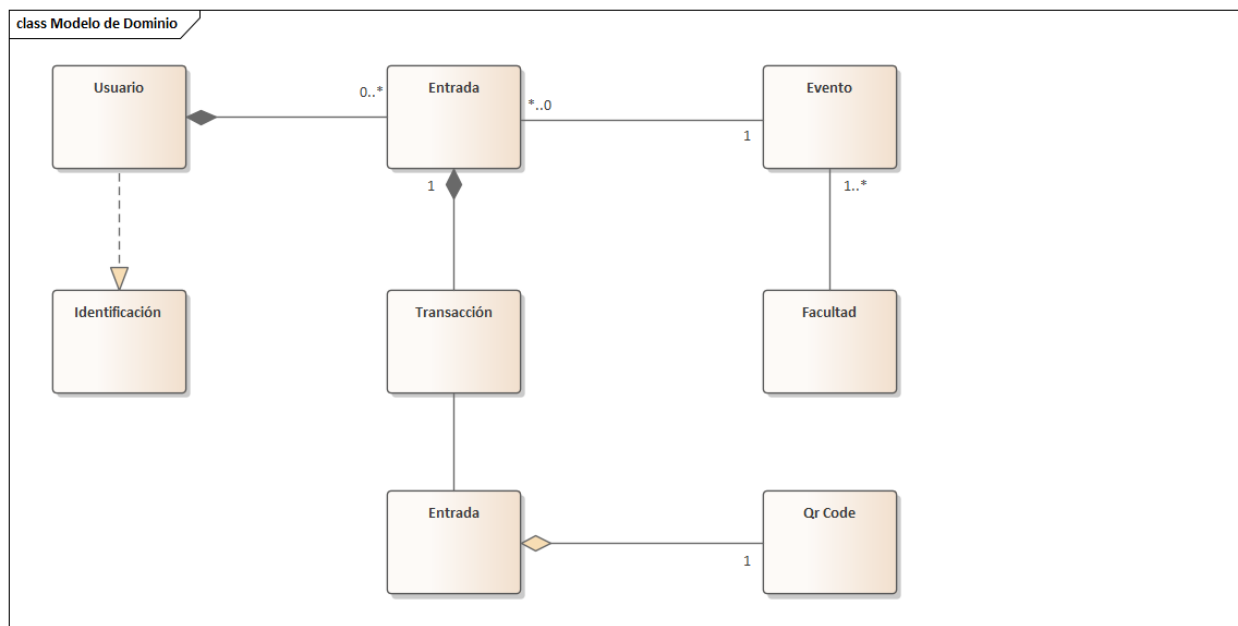


Figura 22. Modelo de dominio

4.3.1. Requisitos funcionales

4.3.1.1. Priorizar casos de uso

Tabla 3. Priorizar casos de uso

Id	Requisito Especifico	Prioridad
RF1	Gestión de eventos	Esencial
RF2	Gestión de entradas	Deseable
RF3	Gestión de facultades	Esencial
RF4	Gestión de categorías de entradas	Esencial
RF5	Generar Entrada QR como Token Único Utilizando Blockchain	Esencial
RF6	Verificación de Identidad mediante Reconocimiento Facial y Documental	Deseable
RF7	Venta y Preventa de Entradas	Esencial
RF8	Transferencia interna de entradas	Opcional
RF9	Escanear Código QR para Validación	Esencial
RF10	Registrar Auditoría de las Entradas	Esencial
RF11	Notificar sobre la compra de entrada	Deseable
RF12	Control de Acceso Durante el Evento	Esencial
RF13	Gestión de suscripciones y tenants	Esencial
RF14	Administración de usuarios y roles	Esencial
RF15	Generación de reportes	Deseable
RF16	Integración de Pagos	Esencial

4.3.1.2. Especificación de requisitos funcionales

RF1. Gestión de eventos

Tabla 4. RF1. Gestión de eventos

RF1	Gestión de eventos	
Descripción	¿Quién?	Organizador
	¿Qué?	Creación y gestión de eventos universitarios.
	¿Por qué?	Configuración de fechas, sectores, categorías y cupos disponibles
Criterios de aceptación		Creación de eventos con todos los campos obligatorios.
		Modificación de eventos sin ventas asociadas.
		El organizador puede eliminar eventos si no están asociados a ninguna entrada vendida.

RF2. Gestión de entradas

Tabla 5. RF2. Gestión de entrada

RF2	Gestión de entradas	
Descripción	¿Quién?	Organizador
	¿Qué?	Definición y administración de entradas
	¿Por qué?	Control de stock y categorización de tipos de acceso.
Criterios de aceptación		Asignación de cantidades por categoría.
		Modificación de cantidades y categorías no vencidas.
		Eliminación de categorías sin asignaciones activas.

RF3. Gestión de facultades

Tabla 6. RF3. Gestión de facultades

RF3	Gestión de facultades	
Descripción	¿Quién?	Administrador universitario
	¿Qué?	Creación y administración de registros de facultades.
	¿Por qué?	Organización y clasificación de eventos por unidad académica.
Criterios de aceptación		Alta de nuevas facultades con nombre y código únicos.
		Edición de datos de facultades existentes.
		El administrador puede eliminar facultades si no están asociadas a eventos activos.

RF4. Gestión de Categorías

RF4	Gestión de categorías	
Descripción	¿Quién?	Administrador universitario
	¿Qué?	Creación y administración de registros de facultades.
	¿Por qué?	Organización y clasificación de eventos por unidad académica.
Criterios de aceptación		Alta de nuevas facultades con nombre y código únicos.
		Edición de datos de facultades existentes.
		El administrador puede eliminar facultades si no están asociadas a eventos activos.

RF5. Generar Entrada QR como Token Único Utilizando Blockchain

Tabla 7. Generar Entrada QR como Token Único Utilizando Blockchain

RF5	Generar Entrada QR como Token Único Utilizando Blockchain	
Descripción	¿Quién?	Organizador
	¿Qué?	Emisión de entradas con hash único en blockchain.
	¿Por qué?	Trazabilidad y autenticidad inmutables
Criterios de aceptación		Generación de hash (Keccak256) por entrada.
		Emisión de código QR asociado a cada hash.
		Registro exitoso de la transacción en la blockchain privada.

RF6. Verificación de Identidad mediante Reconocimiento Facial y Documental

Tabla 8. RF6. Verificación de Identidad mediante Reconocimiento Facial y Documental

RF6	Verificación de Identidad mediante Reconocimiento Facial y Documental	
Descripción	¿Quién?	Universitario
	¿Qué?	Verificar mi identidad mediante reconocimiento facial y documental.
	¿Por qué?	Cumplir con los requisitos de verificación en compras masivas.
Criterios de aceptación		Solicitud de captura de documento y selfie.
		El sistema valida los datos mediante inteligencia artificial.
		La verificación debe ser exitosa para continuar con la compra.

RF7. Venta y preventa de entradas

Tabla 9. RF7 Venta y preventa de entradas

RF7	Venta y preventa de entradas	
Descripción	¿Quién?	Organizador
	¿Qué?	Programación de preventa y venta general de entrada
	¿Por qué?	Habilitar venta anticipada y controlar inventario de entradas.
Criterios de aceptación		Definición de ventana de preventa con fechas de inicio y fin.
		Venta general activa después de la preventa.
		Bloqueo de compra al agotarse el stock.

RF8. Transferencia interna de entradas

Tabla 10. RF8. Transferencia interna de entradas

RF8	Transferencia interna de entradas	
Descripción	¿Quién?	Usuario
	¿Qué?	Transferencia de una entrada a otro usuario registrado.
	¿Por qué?	Facilitar la cesión de entradas entre asistentes.
Criterios de aceptación		Emisión de nueva entrada al receptor con QR válido.
		Revocación de la entrada original.
		Notificación a ambos usuarios sobre la transferencia.

RF9. Escanear código QR para validación

Tabla 11. RF9. Escanear código QR para validación

RF9	Escanear código QR para validación	
Descripción	¿Quién?	Verificador de Acceso
	¿Qué?	Escaneo y comprobación de validez de QR en puerta.
	¿Por qué?	Garantizar acceso únicamente a entradas legítimas.
Criterios de aceptación		Lectura de QR en ≤ 3 s.
		Consulta al smart contract y respuesta “válido” o “usado”.
		Registro de intento de acceso en el log de auditoría.

RF10. Registrar auditoría de las entradas

Tabla 12. RF10. Registrar auditoría de las entradas

RF10	Registrar auditoría de las entradas	
Descripción	¿Quién?	Sistema
	¿Qué?	Registro inmutable de cada emisión y validación de entradas.
	¿Por qué?	Proporcionar trazabilidad completa de transacciones.
Criterios de aceptación		Almacena ID de usuario, hash de entrada y timestamp por transacción.
		Acceso restringido a superadministradores.
		Exportación de logs en CSV o PDF.

RF11. Notificar sobre la compra de entrada

Tabla 12. RF11. Notificar sobre la compra de entrada

RF11	Notificar sobre la compra de entrada	
Descripción	¿Quién?	Sistema
	¿Qué?	Envío automático de correo o push tras la compra.
	¿Por qué?	Confirmar transacción y entregar información de acceso.
Criterios de aceptación		Notificación generada en ≤ 5 s tras la compra.
		Contiene detalles de evento, código QR y enlace de validación.
		Manejo de reintentos en caso de fallo de envío.

RF12. Control de acceso durante el evento

Tabla 13. RF12. Control de acceso durante el evento

RF12	Control de acceso durante el evento	
Descripción	¿Quién?	Verificador de acceso
	¿Qué?	Monitoreo en tiempo real de entradas válidas y denegadas.
	¿Por qué?	Supervisar flujo de asistentes y detectar anomalías.
Criterios de aceptación		Panel muestra conteo de accesos por segundo.
		Actualización automática cada 10 s.
		Indicadores de alertas ante múltiples rechazos consecutivos.

RF13. Gestión de suscripciones y tenants

Tabla 15. RF13. Gestión de suscripciones y tenants

RF13	Gestión de suscripciones y tenants	
Descripción	¿Quién?	Superadministrador (DTIC)
	¿Qué?	Creación, renovación y cancelación de planes y tenants.
	¿Por qué?	Controlar cuotas de eventos y acceso de facultades.
Criterios de aceptación		Planes aplican límites de eventos según tipo (básico, medio, premium).
		Renovación automática envía aviso 7 días antes.
		Cancelación bloquea creación de nuevos eventos.

RF14. Administración de usuarios y roles

Tabla 14. RF14. Administración de usuarios y roles

RF14	Administración de usuarios y roles	
Descripción	¿Quién?	Superadministrador (DTIC)
	¿Qué?	Alta, edición y baja de usuarios (organizador, verificador, superadmin).
	¿Por qué?	Asignar permisos adecuados a cada función.
Criterios de aceptación		Creación de cuenta envía invitación por correo.
		Edición de rol refleja cambios en el menú.

Baja de usuario impide acceso inmediato.
--

RF15. Generación de reportes

Tabla 15. RF15. Generación de reportes

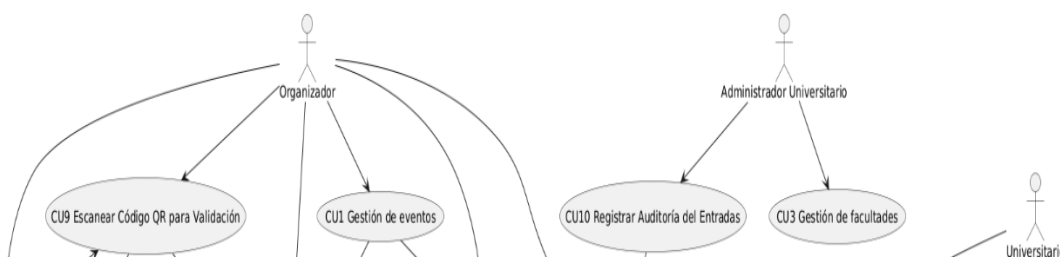
RF15	Generación de reportes	
Descripción	¿Quién?	Organizador y superadministrador
	¿Qué?	Creación de informes de ventas, asistencia y uso de cuotas.
	¿Por qué?	Facilitar análisis y toma de decisiones.
Criterios de aceptación	Filtros por fecha, facultad y evento.	
	Exportación en PDF y XLSX.	
	Tiempo de generación $\leq 5s$ para datasets de hasta 10 000 registros.	

RF16. Integración de pagos

Tabla 18. RF16. Integración de pagos

RF16	Integración de pagos	
Descripción	¿Quién?	Sistema
	¿Qué?	Procesamiento de pagos de suscripciones y entradas mediante Stripe.
	¿Por qué?	Garantizar cobro seguro y registro de transacciones.
Criterios de aceptación	Redirección a Stripe Checkout sin errores.	
	Confirmación de pago recibida vía webhook.	
	Actualización de estado de suscripción o entrada tras pago exitoso.	

4.3.1.3. Diagrama general de casos de uso



4.3.2. Requisitos no Funcionales

Tabla 19. RFN01 Inteligibilidad

Identificación del requerimiento del RNF01	
Nombre requerimiento	Inteligibilidad
Características	- Claridad en la interfaz. - Facilidad de comprensión del sistema.
Descripción requerimiento	El sistema debe ser fácilmente comprensible para usuarios de diferentes niveles de experiencia, asegurando una navegación intuitiva y la comprensión de las funcionalidades ofrecidas.
Prioridad requerimiento	Alta

Tabla 20. RFN02. Desempeño

Identificación del requerimiento del RNF02	
Nombre requerimiento	Desempeño
Características	- Velocidad de respuesta del sistema. - Eficiencia en el procesamiento de datos.
Descripción requerimiento	El sistema debe garantizar un desempeño eficiente, proporcionando respuestas rápidas y procesamiento ágil de datos, incluso en situaciones de carga elevada.
Prioridad requerimiento	Alta

Tabla 21. RFN03 Eficiencia

Identificación del requerimiento del RNF03	
Nombre requerimiento	Eficiencia
Características	- Uso eficiente de recursos del sistema. - Optimización en el uso de la red.

Descripción requerimiento	del	El sistema debe ser eficiente en el consumo de recursos y en el uso de la red para minimizar los tiempos de espera y mejorar la experiencia del usuario.
Prioridad requerimiento	del	Media

Tabla 22. RFN04 Usabilidad

Identificación requerimiento	del	RNF04
Nombre requerimiento	del	Usabilidad
Características		- Facilidad de uso. - Intuitividad de la interfaz de usuario.
Descripción requerimiento	del	El sistema debe ser fácil de usar, con una interfaz intuitiva que permita a los usuarios realizar tareas sin esfuerzo y con mínima formación.
Prioridad requerimiento	del	Alta

Tabla 23. RFN05 Mantenibilidad

Identificación requerimiento	del	RNF05
Nombre requerimiento	del	Mantenibilidad
Características		- Seguir estándares de codificación. - Documentación clara y completa.
Descripción requerimiento	del	El software debe ser fácil de mantener, con un código bien organizado, siguiendo estándares y con documentación que facilite a los desarrolladores entender y dar soporte al sistema.
Prioridad requerimiento	del	

Tabla 24. RFN06 Escalabilidad

Identificación del requerimiento del RNF06	
Nombre requerimiento	Escalabilidad
Características	- Capacidad para manejar un crecimiento en la cantidad de datos y usuarios.
Descripción requerimiento	El sistema debe ser escalable, permitiendo la adición de nuevas funcionalidades y manejando un aumento en la cantidad de datos y usuarios sin degradación significativa del desempeño.
Prioridad requerimiento	Alta

4.3.3. Requisitos de Interfaces

Interfaz de Usuario:

- Debe ser gráfica e intuitiva.
- Adaptable a distintos dispositivos (responsiva).
- Debe tener un diseño moderno y profesional.
- La navegación debe ser sencilla y accesible para usuarios sin conocimientos técnicos.
- Se deben incluir mensajes de error y confirmación claros para mejorar la experiencia del usuario.
- Dashboard de superadmin: gestión de suscripciones, cuotas y reportes.
- Módulo de eventos: listado, creación y edición de eventos.
- Venta de entradas: catálogo de eventos, selección de entradas, proceso de pago.
- Validación de QR: vista simple para escanear y mostrar resultado.

Interfaz con otras aplicaciones:

- Debe contar con una API RESTful para permitir la integración con servicios externos.
- La API debe permitir enviar y recibir datos de forma segura.
- Las respuestas de la API deben ser en formato JSON.
- Se deben considerar posibles integraciones futuras con servicios de terceros (por ejemplo, almacenamiento en la nube o servicios de análisis de audio).

Interfaz de comunicación entre componentes:

- La comunicación debe realizarse mediante el protocolo HTTPS.
- Se deben implementar mecanismos de autenticación (por ejemplo, JWT) y autorización.
- El sistema debe garantizar la seguridad de los datos transmitidos.
- Debe haber manejo de errores en la comunicación entre módulos.

Interfaces externas

- Stripe: Checkout session y webhooks para pagos de entradas y suscripciones.
- Servicio de IA de verificación: POST /api/face-verify con documento y selfie → { match: true/false }.

- Blockchain (Hyperledger Fabric SDK): invocación de smart contracts para emitir y validar transacciones.

4.3.4. Requisitos de Base de Datos

Características generales:

- El sistema debe utilizar un Sistema de Gestión de Base de Datos Relacional (SGBDR), preferentemente PostgreSQL.
- Debe tener un modelo de datos normalizado que evite redundancias.
- La base de datos debe ser escalable para soportar el crecimiento del sistema.

Integridad y consistencia:

- Se deben definir claves primarias para cada tabla.
- Deben establecerse claves foráneas para mantener relaciones entre entidades.
- Se deben aplicar restricciones como:
 - Campos obligatorios (NOT NULL).
 - Restricciones de unicidad (por ejemplo, en el correo electrónico del usuario).
 - Validaciones de formato (por ejemplo, correo válido).

Manejo de archivos y rendimiento:

- La base de datos debe permitir almacenar metadatos de archivos (como rutas o URLs si se almacenan en la nube).
- Se debe contemplar el uso de almacenamiento externo para archivos grandes.
- Se deben usar índices para optimizar las consultas frecuentes.
- Se recomienda implementar copias de seguridad automáticas.

4.3.5. Restricciones de Diseño

Frameworks y lenguajes

- El **frontend** debe implementarse con **Angular ≥ 15** (TypeScript), aprovechando su soporte PWA para acceso offline de entradas.
- El **backend** debe desarrollarse con **NestJS ≥ 9** (TypeScript), estructurado en módulos independientes (EventsModule, TicketsModule, etc.).

Comunicación

- Toda interacción entre cliente y servidor se realizará mediante **APIs RESTful** documentadas con OpenAPI 3.0 (Swagger).
- La serialización de datos empleará JSON con convenciones de nombres **camelCase**.

Blockchain

- La lógica de smart contracts se desarrollará en **Solidity $\wedge 0.8.x$** y se probará/desplegará con **Hardhat ≥ 2.6** .

- En producción, los contratos correrán sobre **Hyperledger Fabric 2.x** o en un **nodo Ethereum** (Sepolia) configurado para pruebas.

Persistencia

- Base de datos relacional: **PostgreSQL ≥ 14** , gestionada con TypeORM para soporte multi-schema (multi-tenant).
- Migraciones automatizadas con **TypeORM Migrations** y respaldos diarios en S3.

Infraestructura

- Contenerización obligatoria con **Docker ≥ 20.10** .
- Despliegue en **AWS EC2** (Docker Compose) con esquemas de autoscaling configurables.
- Variables sensibles gestionadas por **AWS Secrets Manager**.

Estándares de código y calidad

- **ESLint** y **Prettier** para formateo y linting en ambos repositorios.
- Componentes de Angular con pruebas unitarias en **Jest** y e2e en **Cypress**.
- Pruebas de integración de smart contracts en Hardhat con cobertura ≥ 80 %.

4.3.6. Requisitos de Rendimiento

Tiempos de respuesta

- Consultas de eventos y tickets (GET /api/v1/events, GET /api/v1/tickets/{hash}): ≤ 2 s bajo carga normal.
- Operaciones CRUD (crear, editar, borrar eventos, entradas o usuarios): ≤ 1 s en promedio.

Concurrencia

- Soporte de 200 usuarios concurrentes por tenant sin degradación perceptible.
- Escalado horizontal automático: al superar el 70 % de uso de CPU en instancias Docker en EC2, se añaden nodos adicionales.

Rendimiento blockchain

- Procesamiento de al menos 2 000 transacciones por segundo (TPS) durante preventas masivas usando capa 2 (zkSync).
- Validación de códigos QR: 100 escaneos por minuto en un solo verificador, sin aumentar la latencia de consulta a la cadena.

Throughput de API

- El backend debe atender ≥ 100 solicitudes por minuto en endpoints críticos (/tickets, /validate).

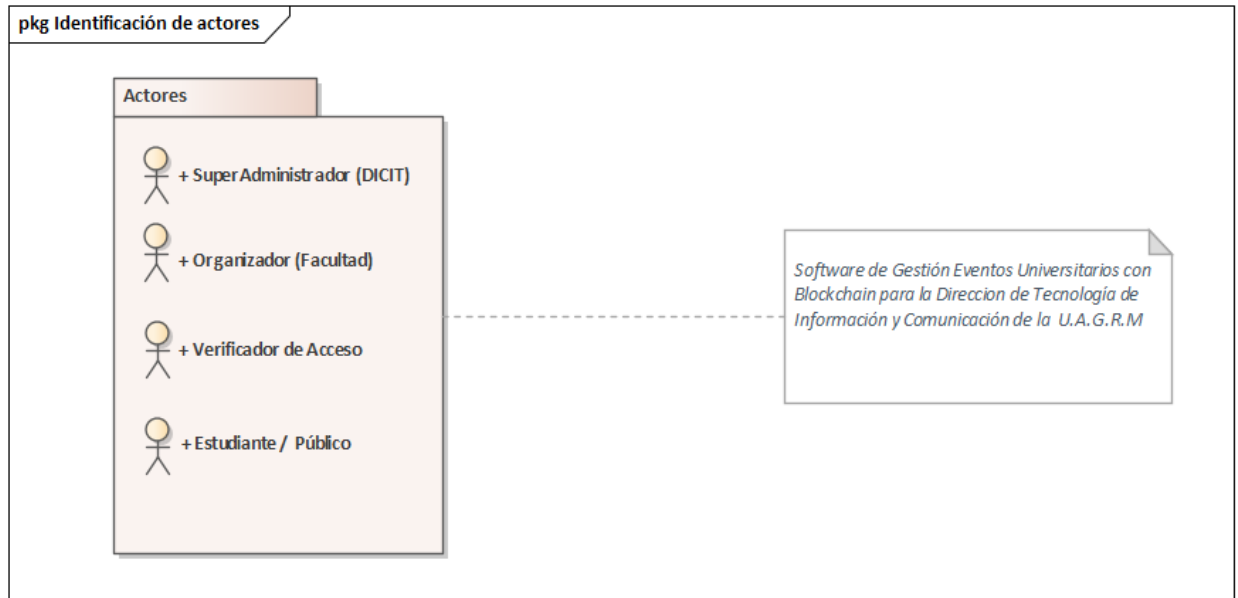
- Para mantener la estabilidad, se implementará caché in-memory (NestJS Cache) y rate limiting (100 req/min por IP).

Escalabilidad

- La arquitectura permitirá añadir nuevas instancias de aplicación sin cambios manuales en la configuración, duplicando la capacidad de usuarios al incrementar el tier de AWS.
- Las colas de procesamiento de pagos y auditoría se gestionarán con Redis Streams para absorber picos de carga.

Capítulo 5 Captura de Requisitos

5.1. Identificación de Actores



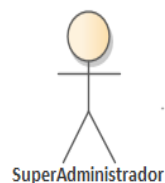
SuperAdministrador (DICIT): Usuario Superadministrador que aprueba las solicitudes de nuevas facultades y renueva o cancela miembros. Provisiona y revoca las cuentas de Organizador y Verificador de Acceso en cada Tenant.

Organizador (Facultad): Usuario Administrativo de una facultad encargado de crear y configura eventos, definir los precios y ventanas de preventa. Generar y administrar entradas digitales y consultar métricas de ventas y asistencias para cada evento.

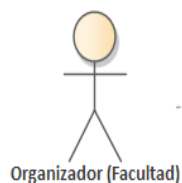
Verificador de Acceso: Usuario designado por la facultad para el control de puertas que accede al PWA para validar y escanear los QR de cada asistente consultando los Smart contract del blockchain.

Estudiante / Público: Usuarios en general que pueden explorar el catalogo de eventos por facultad en la web y comprar entradas a través de pagos por QR.

uc Actores



El Superadministrador de la DTIC accede a un módulo de administración integrado en el mismo front-end de la aplicación, desde donde gestiona de forma centralizada la aprobación de nuevas facultades (tenants), la contratación y renovación de membresías (planes básico, medio o premium) con sus respectivos límites de eventos, y la provisión o revocación de cuentas de Organizadores y Verificadores de Acceso. Además, dispone de vistas protegidas para consultar reportes globales de ventas, validaciones y uso de cuotas, así como del registro completo de auditoría, todo ello sin necesidad de una aplicación separada.



El Organizador representa al personal administrativo de cada facultad. Tras autenticarse, dispone de un panel donde crea y configura sus eventos: establece título, descripción, ubicación, fechas y capacidad; define precios y ventanas de venta; y administra las entradas. Además, genera las entradas digitales (códigos QR vinculados a un hash en blockchain) y puede consultar en tiempo real las métricas de ventas y asistencia de sus eventos, descargando listados o reportes parciales.



Este actor es el encargado de controlar la entrada física a los eventos. Accede a la aplicación de validación (PWA) desde un dispositivo móvil o tablet, escanea el código QR de cada asistente y comprueba su validez consultando el smart contract en la red blockchain. Según el estado del ticket ("válido", "usado" o "inválido"), muestra en pantalla el mensaje correspondiente y registra cada intento en el audit log, garantizando trazabilidad de todos los accesos.



Corresponde a los usuarios finales que navegan el catálogo de eventos disponible en la web pública. Estos pueden filtrar por facultad, fecha o tipo de evento, seleccionar entradas gratuitas o de pago y completar el proceso de compra mediante la pasarela integrada (Stripe). Tras el pago, reciben por correo electrónico o notificación push su código QR, que utilizarán para ingresar al evento. También pueden transferir internamente sus entradas a otros usuarios registrados si lo desean.

5.2. Identificación de Casos de Uso

Identificador	Nombre
CU-01	Gestionar autenticación y recuperación de credenciales
CU-02	Gestionar eventos
CU-03	Gestionar categorías de entradas
CU-04	Gestionar entradas
CU-05	Comprar entradas
CU-06	Verificar identidad en compras masivas
CU-07	Validar QR en puerta
CU-08	Transferir entrada
CU-09	Registrar auditoría de operaciones
CU-10	Notificar evento y compra
CU-11	Monitorear acceso en tiempo real
CU-12	Gestionar suscripciones y tenants

CU-13	Administrar usuarios y roles
CU-14	Generar reportes de ventas y asistencia
CU-15	Procesar pagos de entradas y membresías
CU-16	Renovar suscripción manual
CU-17	Solicitar plan de eventos (facultad envía carta)
CU-18	Aprobar solicitud de plan
CU-19	Configurar plantillas de notificaciones automáticas
CU-20	Desplegar y versionar smart contracts en blockchain privada

5.1. Priorización de los Casos de Uso

Identificador	Nombre	Prioridad
CU-01	Gestionar autenticación y recuperación de credenciales	Alta
CU-02	Gestionar eventos	Alta
CU-03	Gestionar categorías de entradas	Alta
CU-04	Gestionar entradas	Alta
CU-05	Comprar entradas	Alta
CU-06	Verificar identidad en compras masivas	Media
CU-07	Validar QR en puerta	Alta
CU-08	Transferir entrada	Baja
CU-09	Registrar auditoría de operaciones	Alta
CU-10	Notificar evento y compra	Media
CU-11	Monitorear acceso en tiempo real	Alta
CU-12	Gestionar suscripciones y tenants	Alta
CU-13	Administrar usuarios y roles	Alta
CU-14	Generar reportes de ventas y asistencia	Media
CU-15	Procesar pagos de entradas y membresías	Alta
CU-16	Renovar suscripción manual	Media
CU-17	Solicitar plan de eventos (facultad envía carta)	Media
CU-18	Aprobar solicitud de plan	Alta
CU-19	Configurar plantillas de notificaciones automáticas	Baja

CU-20	Desplegar y versionar smart contracts en blockchain privada	Media
-------	---	-------

5.2. Detalle de Casos de Uso